

$SU(2)$ Gauge Theory in Classical Mechanics in $\mathcal{UD}$

Brian Beckman

Mar 2026

Abstract

Here we present geometric equations of motion in the gauge group $SU(2)$ for the inverted spherical pendulum and the axially symmetrical top. The derivation is exactly parallel to our prior geometric equations in *$SO(3)$ Gauge Theory for Classical Mechanics in $\mathcal{UD}$* , only this time avoiding gimbal lock via better gauge choice. Both derivations are exceptionally concise, expressing the equations and solutions as geodesics on manifolds. It is remarkable that a full description of dynamics arises, yet again, from differential geometry alone.

We finish with a side-by-side animation showing numerical equivalence, aside from gimbal lock in the $SO(3)$ case, which we can provoke by finding the $SO(3)$ poles with the **Manipulate** sliders. That is how we demonstrate superiority of the $SU(2)$ gauge choice over $SO(3)$.

We study $SU(2)$ here in an intuitive setting of classical mechanics so that we will be better equipped for transition to quantum physics. Instead of studying two abstractions at once in the quantum setting— $SU(2)$ and spinors—we will have only spinors to study. This notebook contains a full chapter on the theory of $SU(2)$, requiring only basic University mathematics, to further lower barriers to understanding.

Preliminaries

Initialize the Relativity Toolkit

Optional: Quit for a Fresh Kernel

- *Note, if you call **Quit** in a notebook, evaluation may hang. If you want to “Evaluate Notebook,” mark the following two cells “Evaluatable” in the Cell→Property menu, call **Quit** and **FrontEndTokenExecute** once each, then comment out those two cells, or mark them unevaluatable again. Alternatively, you can quit the kernel via the Evaluation menu (last item). At that point, you can “Evaluate Notebook” in a clean environment, or evaluate cells individually.*

```
Quit[];
```

```
In[*]:= FrontEndTokenExecute["DeleteGeneratedCells"]
```

Download the Code

```
In[1]:= getToolkit[branch_] :=
  With[
    {base = "https://raw.githubusercontent.com/rebcabin/RelativityToolkit/" <>
      branch <> "/",
      bust = "?t=" <> ToString[Round[AbsoluteTime[]]]},
    {rules = base <> "RelativityToolkit.wl" <> bust},
    Get[rules]];
getToolkit["v1.13.1"];
» Relativity Toolkit version : 1.13.1
» Relativity Connection Symbol :  $\Gamma$ 
```

Setup

Global Parameters

```
In[3]:= ClearAll[e1, e2, e3, o, g, plotRg];
e1 = {1, 0, 0}; e2 = {0, 1, 0}; e3 = {0, 0, 1};
o = {0, 0, 0};
g = 9.81;
plotRg = {-1.25, 1.25};

ClearAll[x, y, z, u, v, w, t];

ClearAll[tLim];
tLim = 200;
```

Utility Function

```
In[11]:= radPerSec[rpm_] :=  $\frac{2 \cdot \pi}{60.} * rpm;$ 
```

Logic Utility Functions

```
In[12]:= ClearAll[proof];
proof[v_] /; BooleanQ[v] := (If[v,
  Print[Style["QED■", StandardGreen, Bold, 18]],
  Print[Style["FAILED PROOF■", StandardRed, Bold, 18]]]; v);
proof[___] := Print[Style["INVALID INPUT TO PROOF■", StandardYellow, Bold, 18]];
```

Predicates

```
In[15]:= ClearAll[TracelessQ, HermitianQ, AntiHermitianQ, UnitaryQ];
TracelessQ[m_] := Tr[m] === 0;
HermitianQ[m_] := (ComplexExpand[m†] === ComplexExpand[m]);
AntiHermitianQ[m_] := (ComplexExpand[m†] === ComplexExpand[-m]);
UnitaryQ[m_] := (m†.m === m.m† === IdentityMatrix[2]);
```

Graphical and Physical Elements

```
In[20]:= ClearAll[rig];
With[{l = 1.2, mass = 1.0, d = 2.4, r = 0.02},
  {baton = Cylinder[{0, 0, -l}, {0, 0, +l}], r},
  batonCenterBodyFrame = {0, 0, l}},
rig["graphics primitives"] = baton;
rig["moment of inertia"] =
  With[{Ib = mass * (MomentOfInertia[baton] / Volume[baton])},
    {Ibi = Inverse@Ib},
    {Ib, Ibi}];
rig["cb"] := batonCenterBodyFrame;
rig["half length"] := l;
rig["mass"] := mass;];
ClearAll[jack];
jack[0, opacity_ : 0.1, diameter_ : 0.01] := {
  Opacity[opacity],
  {Red, Arrow[Tube[{0, e1}], diameter]},
  {Darker[Green], Arrow[Tube[{0, e2}], diameter]},
  {Lighter[Blue], Arrow[Tube[{0, e3}], diameter]},
  {Lighter[Cyan], Arrow[Tube[{0, -e1}], diameter]},
  {Magenta, Arrow[Tube[{0, -e2}], diameter]},
  {Yellow, Arrow[Tube[{0, -e3}], diameter]}};

ClearAll[nfm, vfm, font, myFont];
myFont[color_, size_ : 18] :=
  Style[#, color, Bold, size, FontFamily → "Courier New"] &;
nfm[num_, dig_ : 10, dec_ : 4] := ToString[NumberForm[Chop@num, {dig, dec}]];
vfm[vec_] := ToString[
  NumberForm[#, {7, 4}, NumberSigns → {"-", " "}, NumberPadding → {" ", "0"}] & /@
  Chop@vec];
font = myFont[Lighter[Blue], 12];
```

Chapter I: $SO(3)$ Recap

In this chapter, we reprise the derivations from *$SO(3)$ Gauge Theory for Classical Mechanics in $\mathcal{UD}$* , minus the prose explanations, for two reasons:

1. To emphasize the brevity of the geometric derivation.
2. To set up for the final, side-by-side animation of this notebook.

$SO(3)$ Geometrical Equations of Motion

```

In[29]:= ClearAll[coords3, vels3, v3, r3, I3, I1, cz, m, t, V];
coords3 = {δ, η, ψ};
metric3 =  $\begin{pmatrix} I1 & 0 & 0 \\ 0 & (I1 \cos[\delta]^2 + I3 \sin[\delta]^2) & I3 \sin[\delta] \\ 0 & I3 \sin[\delta] & I3 \end{pmatrix}$ ;
g3 = MakeIndexer[metric3, coords3];
ig3 = MakeIndexer[Inverse@metric3, coords3];

vels3 = (c ↦ c'[t]) /@ coords3;
v3 = MakeIndexer[vels3, coords3];

V = m g cz Cos[δ] Cos[η];

r3[μ_, ν_, λ_] := ChristoffelsFromMetricIndexer[g3, ig3, μ, ν, λ, coords3];
(ad3 = AbsoluteDerivativeFromConnectionIndexer[v3, r3, t, coords3] // FullSimplify);
(pf3 = PotentialScalarFieldFromMetricIndexer[V, ig3, t, coords3] // FullSimplify);

(eqns3 = MapThread[Equal, {ad3, pf3}] // FullSimplify);

```

Chapter II: $SU(2)$ Theory

$SU(2)$ normally appears in a Physics curriculum for the first time in quantum mechanics, applied to spinors. However, its origins are in classical mechanics, where it appears as quaternions, specifically to resolve gimbal lock. It took a very long time after the realization that no choice of Euler angles can prevent gimbal lock to synthesize a concise theory of rotations in classical mechanics. $SU(2)$ is the culmination of a long chain of discoveries by Euler, Rodrigues, Hamilton, Cayley, Klein, Galois, Lie, and others that go through several areas of mathematics: topology, geometry, differential equations, and algebra.

The surprising outcome is that, despite appearances, 3D space requires two full rotations— 720° —to restore a system to its original topological state, avoiding twist relative to the environment. A single rotation of 360° leaves a system in a “twisted” state, requiring another 360° —in the same direction—to undo. This fact can be demonstrated by loosely strapping an apparatus to a lab bench and then turning the apparatus about a vertical axis. There are numerous other demonstrations such as the Dirac belt trick and the Indonesian candle dance. There is nothing quantum or abstract about this fact.

We cannot see this fact directly in $SO(3)$ because the Euler angles of the original state and of the twisted state are the same. However, in $SU(2)$, they are of opposite algebraic signs. The representations of the original and twisted states in $SU(2)$ can be smoothly connected by a curve—a smooth trajectory—entirely in $SU(2)$. Such is not possible in $SO(3)$, where to get from one state to the other, we must pass through a coordinate singularity, resulting in gimbal lock. $SO(3)$ is literally only half the story of rotation in 3D. Another view is “double cover:” that a single rotation representation in $SO(3)$ has exactly two

representations in $SU(2)$.

There is a lot of material in this chapter, but the payoff in the next chapter is a concise and elegant geometric derivation of the equations of motion as geodesics in $SU(2)$, exactly parallel to the derivation vetted previously for $SO(3)$.

Summary: The Lie Group $SU(2)$ and its Lie Algebra, $\mathfrak{su}(2)$

This summary presents the basic facts necessary to understand the building blocks of the derivations: the metric tensor, the potential field, the objects put to rotation, and the rotation operators themselves. Most of the facts have proofs in the remainder of this chapter. The proofs require only basic University mathematics. There are many short reminders of even more basic facts, intended to ease understanding.

In[138]:=

```
Grid[{
  Style[#, Bold, 14] & /@ {"Concept", "Properties", "Basis", "Space", "Role"},
  {"State", "Hermitian, Tr=0", "{ $\sigma_k$ ", " $\mathfrak{i} \mathfrak{su}(2) \equiv \mathbb{R}^3$ ", "The Vector being rotated"},
  {"Generator", "Anti-Herm, Tr=0", "{ $-\mathfrak{i}\sigma_k$ ", " $\mathfrak{su}(2)$ ",
    "Angular Velocity / Twist Vector"}, {"Rotation Q(t)", "Unitary, Det=1",
    "exp( $\mathfrak{su}(2)$ )", "SU(2)", "Rot. Operator / Path in Group"}, {"Versor q(t)",
    "Norm = 1", "{1,  $\mathfrak{i}$ ,  $\mathfrak{j}$ ,  $\mathfrak{k}$ ", " $H_1 \equiv S^3$ ", "Quaternion Representation"}},
  Frame → All, Spacings → {1.5, 1.5}, Alignment → Left, Background → {None, {Gray}}]
```

Out[138]=

Concept	Properties	Basis	Space	Role
State	Hermitian, Tr=0	$\{\sigma_k\}$	$\mathfrak{i} \mathfrak{su}(2) \equiv \mathbb{R}^3$	The Vector being rotated
Generator	Anti-Herm, Tr=0	$\{-\mathfrak{i}\sigma_k\}$	$\mathfrak{su}(2)$	Angular Velocity / Twist Vector
Rotation Q(t)	Unitary, Det=1	exp($\mathfrak{su}(2)$)	SU(2)	Rot. Operator / Path in Group
Versor q(t)	Norm = 1	$\{1, \mathfrak{i}, \mathfrak{j}, \mathfrak{k}\}$	$H_1 \equiv S^3$	Quaternion Representation

Lie Group $SU(2)$

DEFINITION: The **Lie Group** $SU(2)$ is the group of all 2×2 complex, unitary matrices with unit determinant.

- *Unitary ($A^\dagger = A^{-1}$), for complex matrices, is the analogue of orthogonal ($A^T = A^{-1}$), for real matrices. Orthogonal matrices are familiar as representations of rotations.*

- “A-dagger,” $A^\dagger$, is the complex-conjugate transpose of the matrix A . The dagger operation is available in Wolfram as **ConjugateTranspose**, and in postfix notation in a notebook via the keystrokes `[ESC]ct[ESC]`.
- Unitarity guarantees that eigenvalues have absolute value 1.
- Unit determinant implies a **unit norm** for the quaternion representation.

We interpret $SU(2)$ as a group of rotation operators. They are isomorphic to versors—Hamilton’s quaternions with unit norm.

- A **group** is a set with an associative binary operation, an identity element, and an inverse for every element.

Adjoint Action

In classical mechanics, a rotation operator, $Q \in SU(2)$, acts on v , a 2×2 complex representation of a real 3-vector, via **adjoint action**:

$$v \rightarrow Q.v.Q^\dagger, \quad Q \in SU(2) \tag{1}$$

- In quantum physics, $SU(2)$ operators act on spinors by direct action: $|\xi\rangle \rightarrow Q|\xi\rangle$.

v is 2×2 , complex, traceless, and Hermitian (**self-adjoint**, $v^\dagger = v$)—the expansion in Pauli basis (see below) of an ordinary, real 3-vector.

- v is not a member of $SU(2)$.
- $v.Q^\dagger$ is not a spinor, though it can be related to spinors via Clifford algebra (TODO).
- v is also not a member of $\mathfrak{su}(2)$, whose members are anti-Hermitian.
- (ancillary fact): The eigenvalues of v , as with any Hermitian matrix, are real.

In the equations of motion, we model the baton as one of these vectors.

Manifolds, Curves, Tangents, Geodesics, Trajectories

$SU(2)$, like any Lie group, is also a differentiable manifold. We can set up parameterized curves, derive tangent vectors, calculate geodesics, and so on. That is how we model dynamics of the pendulum and top: as trajectories of rotations along geodesics in $SU(2)$.

- $SO(3)$, the set of 3×3 real, orthogonal matrices with unit determinant, is also a Lie Group, but, as we have seen in [SO\(3\) Gauge Theory](#), it is impossible to guarantee that a trajectory in $SO(3)$ will not gimbal-lock, i.e., $SO(3)$ cannot be covered by one non-singular chart.

Lie Algebra $\mathfrak{su}(2)$

DEFINITION: The **Lie algebra** $\mathfrak{su}(2)$ is the unique tangent space $T_{\mathbb{I}_2} SU(2)$, at the identity $\mathbb{I}_2$ of $SU(2)$. It contains tangent vectors to curves in $SU(2)$, transported by the group’s operation from anywhere within $SU(2)$ back to the origin of $SU(2)$. $\mathfrak{su}(2)$ is thus a vector space.

- $\mathfrak{su}(2)$ is trivially a group under vector addition, but that fact is uninteresting. The primary dynamical operation in $\mathfrak{su}(2)$ is the Lie Bracket, $[A, B] = AB - BA$, a model of the ordinary cross product. $\mathfrak{su}(2)$ is **not a group** under this operation because there is no identity and no inverse (TODO: elaborate).

$\mathfrak{su}(2)$ is also the set of all 2×2 complex, traceless, anti-Hermitian ($A^\dagger = -A$) matrices. The following four propositions prove this fact:

1. Every element of $\mathfrak{su}(2)$ is of the form $\dot{Q}(t).Q(t)^\dagger$, where $Q(t)$ is a smooth curve in $SU(2)$ and $\dot{Q}(t)$ is tangent to the curve at the point $Q(t)$.
 - 1.1. **IMPORTANT:** This identification of elements as $\dot{Q}(t).Q(t)^\dagger$ is conventional and presupposes the inertial “space frame” for mechanics. This convention is convenient and standard for the proofs presented in this chapter, but not for computation, where we go to the “body frame” in the following chapters, and elements are $Q(t)^\dagger.\dot{Q}(t)$.
2. Unitarity of $SU(2)$ implies that elements of $\mathfrak{su}(2)$ are anti-Hermitian.
3. Unit determinants for elements of $SU(2)$ implies that elements of $\mathfrak{su}(2)$ are traceless.
4. Any traceless, anti-Hermitian matrix A has the form $\dot{Q}(t).Q(t)^\dagger$ for some curve in $SU(2)$, and therefore is in $\mathfrak{su}(2)$.

Proposition 1

◆ Every element of $\mathfrak{su}(2)$ is of the form $\dot{Q}(t).Q(t)^\dagger$, where $Q(t)$ is a smooth curve in $SU(2)$ and $\dot{Q}(t)$ is tangent to the curve at the point $Q(t)$.

PROOF: Let $Q(t)$ be the trajectory of the baton (or of any other system), a smooth curve in $SU(2)$. At any specific time t , $\dot{Q}(t)$ is tangent to the curve at the point $Q(t)$, in the tangent space $T_{Q(t)} SU(2)$. Now consider another curve $\gamma(s) = Q(t+s).Q(t)^\dagger$ (no dot on the first Q), intended to bring $Q(t+s)$ to the origin via group multiplication by $Q(t)^\dagger$.

- $\gamma(s=0)$ is the identity, $\mathbb{I}_2$, of $SU(2)$: $\gamma(0) = Q(t+0).Q(t)^\dagger = Q(t).Q(t)^\dagger = \mathbb{I}_2$.
- Every point along $\gamma(s)$ is in the group $SU(2)$ because $SU(2)$ is closed under multiplication and the inverse $Q(t)^\dagger = Q(t)^{-1}$ is also in $SU(2)$.
 - The inverse of $Q(t)$ is $Q(t)^\dagger$ by unitarity. The inverse of any element of $SU(2)$ is also in $SU(2)$ because $SU(2)$ is a group.
- The tangent vector of $\gamma(s)$ at the identity of $SU(2)$ is

$$\left. \frac{d\gamma(s)}{ds} \right|_{s=0} = \left. \frac{\partial}{\partial s} Q(t+s).Q(t)^\dagger \right|_{s=0} = \left. \dot{Q}(t+s).Q(t)^\dagger \right|_{s=0} = \dot{Q}(t).Q(t)^\dagger = \mathbb{I}_2 \quad (2)$$

```
In[42]:= proof@Module[{Q, Qdot, X, Y},
  (* arbitrary curve in SU(2); take on faith for now *)
  Q[t_] := { x[t] + I y[t] u[t] + I v[t],
             -u[t] + I v[t] x[t] - I y[t] };
  Qdot[t_] := D[Q[t], t];
  Echo[X = (D[Q[s+t].Q[t]^\dagger, s] /. s -> 0) // ComplexExpand] // Short,
  "∂_s (Q(t+s).Q(t)^\dagger) |_{s=0} := ";
  Echo[Y = (Qdot[t].Q[t]^\dagger) // ComplexExpand] // Short, "Qdot[t].Q[t]^\dagger := ";
  Echo[X == Y, "∂_s (Q(t+s).Q(t)^\dagger) |_{s=0} == Qdot[t].Q[t]^\dagger := "];
```


» $\partial_s(Q(t+s).Q(t)^\dagger)|_{s=0} : \{\{\langle\langle 1 \rangle\rangle\}, \{-x[t] u'[t] + \langle\langle 4 \rangle\rangle + i (\langle\langle 1 \rangle\rangle), \langle\langle 1 \rangle\rangle\}\}$

» $\dot{Q}(t).Q(t)^\dagger : \{\{\langle\langle 1 \rangle\rangle\}, \{-x[t] u'[t] + \langle\langle 4 \rangle\rangle + i (\langle\langle 1 \rangle\rangle), \langle\langle 1 \rangle\rangle\}\}$

» $\partial_s(Q(t+s).Q(t)^\dagger)|_{s=0} == \dot{Q}(t).Q(t)^\dagger : \text{True}$

QED ■

Proposition 2

◆ Unitarity of elements of $SU(2)$ implies that elements of $\mathfrak{su}(2)$ are anti-Hermitian.

PROOF: Differentiate the unitarity condition of an arbitrary element $Q \in SU(2)$ with respect to time:

$$\begin{aligned} \frac{d}{dt}(Q.Q^\dagger) &= 1 \\ \dot{Q}.Q^\dagger + Q.\dot{Q}^\dagger &= 0 \\ \dot{Q}.Q^\dagger &= -Q.\dot{Q}^\dagger = -(\dot{Q}.Q^\dagger)^\dagger \end{aligned} \tag{3}$$

I.e., $\dot{Q}.Q^\dagger$ is anti-Hermitian.

```
In[43]:= proof@Module[{Q, Qdot, X, minusXdagger, unitNormConstraint, duncdt},
  (* arbitrary member of SU(2) on a curve; take on faith for now *)
  Q[t] = { x[t] + i y[t] u[t] + i v[t],
            -u[t] + i v[t] x[t] - i y[t] };
  Echo[Q[t].Q[t]^\dagger // ComplexExpand // MatrixForm, "Q.Q^\dagger :"];
  unitNormConstraint =
    Echo[(Q[t].Q[t]^\dagger // ComplexExpand)[[1, 1]] == 1, "unit norm constraint :"];
  duncdt = Echo[D[unitNormConstraint, t], "d(unit norm constraint)/dt :"];
  Qdot[t] = D[Q[t], t];
  Echo[
    (X = Qdot[t].Q[t]^\dagger // ComplexExpand // FullSimplify[#, Assumptions -> duncdt] &) //
    Short, "\dot{Q}.Q^\dagger :"];
  Echo[
    (minusXdagger = -X^\dagger // ComplexExpand // FullSimplify[#, Assumptions -> duncdt] &) //
    Short, "-(\dot{Q}.Q^\dagger)^\dagger :"];
  Echo[X == minusXdagger, "\dot{Q}.Q^\dagger == -(\dot{Q}.Q^\dagger)^\dagger :"]];
```

```

» Q.Q†:  $\begin{pmatrix} u[t]^2 + v[t]^2 + x[t]^2 + y[t]^2 & 0 \\ 0 & u[t]^2 + v[t]^2 + x[t]^2 + y[t]^2 \end{pmatrix}$ 
» unit norm constraint:  $u[t]^2 + v[t]^2 + x[t]^2 + y[t]^2 == 1$ 
» d(unit norm constraint)/dt:  $2 u[t] u'[t] + 2 v[t] v'[t] + 2 x[t] x'[t] + 2 y[t] y'[t] == 0$ 
»  $\dot{Q}.Q^\dagger$ :  $\{\{i (-v[t] u'[t] + \ll 1 \gg - \ll 1 \gg + x[t] y'[t])\}, \ll 1 \gg\}, \ll 1 \gg\}$ 
»  $-(\dot{Q}.Q^\dagger)^\dagger$ :  $\{\{i (-v[t] u'[t] + \ll 1 \gg - \ll 1 \gg + x[t] y'[t])\}, \ll 1 \gg\}, \ll 1 \gg\}$ 
»  $\dot{Q}.Q^\dagger == -(\dot{Q}.Q^\dagger)^\dagger$ : True
QED ■

```

Proposition 3

◆ Unit determinants for elements of $SU(2)$ implies that elements of $\mathfrak{su}(2)$ are traceless.

PROOF: By Jacobi's formula for the derivative of a determinant

$\det(Q(t)) = 1$ (by hypothesis)

$$\frac{d}{dt} \det(Q(t)) = 0$$

$$\frac{d}{dt} \det(Q(t)) = (\det(Q(t)) = 1) \cdot \text{Tr} \left(Q^{-1}(t) \cdot \dot{Q}(t) = Q(t)^\dagger \cdot \dot{Q}(t) \right)$$

(4)

$$\Rightarrow \text{Tr} \left(Q(t)^\dagger \cdot \dot{Q}(t) \right) = \text{Tr} \left(\dot{Q}(t) \cdot Q(t)^\dagger \in \mathfrak{su}(2) \right) = 0$$

```

In[44]:= proof@Module[{Q, X, Qdot, Dconstraint, tr},
  (* arbitrary member of SU(2) on a curve; take on faith for now *)
  Q[t] =  $\begin{pmatrix} x[t] + i y[t] & u[t] + i v[t] \\ -u[t] + i v[t] & x[t] - i y[t] \end{pmatrix}$ ;
  (* Determinant is square norm and equal to 1 *)
  Echo[Det[Q[t]], "Det[Q] :"];
  Dconstraint = Echo[D[Det[Q[t]], t] == 0, "Dconstraint :"];
  Qdot[t] = D[Q[t], t];
  Echo[(X = (Qdot[t].Q[t]†) // ComplexExpand) // Short,
    "X =  $\dot{Q}[t].Q[t]^\dagger \in \mathfrak{su}(2)$  :"];
  Echo[tr = FullSimplify[Tr[X], Assumptions → Dconstraint], "Tr[X] :"];
  tr == 0];

```

```

» Det[Q]:  $u[t]^2 + v[t]^2 + x[t]^2 + y[t]^2$ 
» Dconstraint:  $2 u[t] u'[t] + 2 v[t] v'[t] + 2 x[t] x'[t] + 2 y[t] y'[t] == 0$ 
»  $X = \dot{Q}[t].Q[t]^\dagger \in \mathfrak{su}(2)$ :  $\{\{ \ll 1 \gg \}, \{-x[t] u'[t] + \ll 4 \gg + i (\ll 1 \gg), \ll 1 \gg\}\}$ 
» Tr[X]: 0

```

QED ■

Proposition 4

◆ Any traceless, anti-Hermitian matrix A has the form $\dot{Q}(t).Q(t)^\dagger$ for some curve $Q(t)$ in $SU(2)$, and therefore A is in $\mathfrak{su}(2)$.

PROOF: Let $Q(t) = e^{At}$ describe a curve in $SU(2)$. Then $\dot{Q}(t) = A e^{At}$. Because the Taylor series of the matrix exponential has only real, scalar powers of t , the conjugate-transpose operation, † , lifts into the exponent: $Q(t)^\dagger = (e^{At})^\dagger = e^{A^\dagger t}$. Because A is anti-Hermitian, $Q(t)^\dagger = e^{-At}$. Now, $\dot{Q}(t).Q(t)^\dagger = A e^{At} e^{-At} = A e^0 = A$ because At and $-At$ commute (condition for adding exponents in a matrix exponential). Therefore A has the form $\dot{Q}.Q^\dagger$ and must be in $\mathfrak{su}(2)$ by Proposition 1.

```
In[45]:= proof@Module[{A, Q, Qdot, X, Y},
  (* arbitrary traceless, anti-Hermitian matrix *)
  A =  $\begin{pmatrix} \text{I} y & u + \text{I} v \\ -(u + \text{I} v)^* & -\text{I} y \end{pmatrix}$ ;
  Echo[TracelessQ[A] && AntiHermitianQ[A], "A traceless && anti-Hermitian :"];
  Q[t] = MatrixExp[A t];
  Qdot[t] = D[Q[t], t];
  X = MatrixExp[A t]^\dagger // ComplexExpand;
  Y = MatrixExp[A^\dagger t] // ComplexExpand;
  With[{zero = {{0, 0}, {0, 0}}}, {
    expIdentity = Echo[
      FullSimplify[
        X - Y == 0, Assumptions -> {y, u, v, t} \in Reals],
      "(e^{At})^\dagger == e^{A^\dagger t} :"],
    prop4 = Echo[
      FullSimplify[
        ComplexExpand[
          Qdot[t].Q[t]^\dagger - A == zero],
        Assumptions -> {y, u, v, t} \in Reals], "prop4 :"],
    prop4]
  ];
  » A traceless && anti-Hermitian : True
  » (e^{At})^\dagger == e^{A^\dagger t} : True
  » prop4 : True
```

QED ■

Pauli Matrices as Hermitian Basis for 3-Vectors

Any linear combination of the three Pauli matrices *with real coefficients* is traceless and Hermitian. Therefore, any real 3-vector, $(x, y, z) \in \mathbb{R}^3$, can be represented as a 2×2 , traceless, Hermitian matrix, and vice versa, as follows:

```
In[46]:= ClearAll[σ, σVec, id];
σVec = ConstantArray[0, 3];
id = IdentityMatrix[2];
σVec[[1]] = σ[1] = PauliMatrix[1];
σVec[[2]] = σ[2] = PauliMatrix[2];
σVec[[3]] = σ[3] = PauliMatrix[3];
Echo[MatrixForm /@ σVec, "Pauli matrices :"];
With[{v = {x, y, z}}, Echo[v.σVec // MatrixForm,
  "any vector expanded in Pauli basis :"]];
```

» Pauli matrices : $\left\{ \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}, \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix}, \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix} \right\}$

» any vector expanded in Pauli basis : $\begin{pmatrix} z & x - i y \\ x + i y & -z \end{pmatrix}$

Traceless and Hermitian

Pauli matrices are obviously traceless, plus they are Hermitian: each is equal to its conjugate transpose. “Hermitian” is the analogue of “symmetric” amongst real matrices and implies real eigenvalues.

```
In[54]:= proof@Echo[And@@ (HermitianQ /@ σVec), "Pauli matrices are Hermitian :"];
```

» Pauli matrices are Hermitian : True

QED ■

```
In[55]:= Echo[(Eigenvalues /@ σVec) // MatrixForm, "Eigenvalues of Pauli Matrices :"];
```

» Eigenvalues of Pauli Matrices : $\begin{pmatrix} -1 & 1 \\ -1 & 1 \\ -1 & 1 \end{pmatrix}$

The vector components cannot, in general, be complex. For example, $i\sigma_1$ is not Hermitian, i.e., is not equal to its conjugate transpose:

```
In[139]:= With[{ImQ = {i, 0, 0}},
  {counterexample = Echo[ImQ.σVec // MatrixForm, "counterexample {i,0,0}.σ :"],
  proof[Not@HermitianQ[counterexample]]];
```

» counterexample {i,0,0}.σ : $\begin{pmatrix} 0 & i \\ i & 0 \end{pmatrix}$

QED ■

As noted in the *Summary* above, we will subject a 3-vector representative v (traceless, Hermitian) to a rotation by adjoint action, $Q.v.Q^\dagger$.

Unitary (Ancillary Fact)

Pauli matrices are also unitary, meaning that every Pauli matrix satisfies $\sigma_i^2 = \mathbb{I}_2$ and that all eigenvalues have absolute value 1.

```
In[57]:= proof@Echo[And@@ (UnitaryQ /@ σVec), "Pauli matrices are Unitary :"];
```

» Pauli matrices are Unitary : True

QED ■

- Therefore, the Pauli matrices, as unitary operators, are reflections. Every rotation in 3D is a sequence of two reflections (TODO: elaborate)

$-i\vec{\sigma}$ as a Basis for $\mathfrak{su}(2)$

Any traceless, anti-Hermitian matrix—member of $\mathfrak{su}(2)$ —can be expanded in $-i\vec{\sigma}$:

```
In[58]:= proof@Module[{A, basis, coeffs},
  (* arbitrary traceless, anti-Hermitian matrix *)
  A =  $\begin{pmatrix} i w & u + i v \\ -(u + i v)^* & -i w \end{pmatrix}$ ;
  basis = -i σVec;
  (* solve for these coefficients in the -iσ basis *)
  coeffs = {x, y, z};
  Solve[coeffs.basis == A, coeffs, Assumptions → {u, v, w} ∈ Reals][[1]] ==
  Echo[{x → -v, y → -u, z → -w},
    "Coefficients of expansion for any traceless, anti-Hermitian matrix :"]];
```

» Coefficients of expansion for any traceless, anti-Hermitian matrix : {x → -v, y → -u, z → -w}

QED ■

Conversely, any linear combination of the $-i\vec{\sigma}$ matrices with real coefficients is traceless and anti-Hermitian, therefore in $\mathfrak{su}(2)$:

```
In[59]:= Echo[With[{q = {q1, q2, q3}},
  (* ComplexExpand injects the assumption that the coefficients are real *)
  -i q.σVec // ComplexExpand] // MatrixForm,
  "traceless, anti-Hermitian, in su(2) :"];
» traceless, anti-Hermitian, in su(2) :  $\begin{pmatrix} -i q_3 & -i q_1 - q_2 \\ -i q_1 + q_2 & i q_3 \end{pmatrix}$ 
```

```
In[60]:= proof@Echo[With[
  {q = {q1, q2, q3}},
  {vp = -i q.σVec},
  AntiHermitianQ[vp] && TracelessQ[vp]],
  "vector part is in su(2) :"];
```

» vector part is in su(2) : True

QED ■

Quaternions as a Coordinate Chart in $S^3 \subset \mathbb{R}^4$ for $SU(2)$

$SU(2)$ operators are faithfully represented by polynomials in $\{\mathbb{I}_2\} \cup (-i\vec{\sigma})$. The four *real-valued* coeffi-

cients (q_0, q_1, q_2, q_3) are constrained to unit norm and constitute the components of a **unit quaternion**, aka a **versor**:

$$Q \equiv q_0 \mathbb{I}_2 - i(q_1 \sigma_1 + q_2 \sigma_2 + q_3 \sigma_3) \equiv q_0 \mathbb{I}_2 - i \vec{q} \cdot \vec{\sigma} \quad (5)$$

where $(q_0^2 + q_1^2 + q_2^2 + q_3^2) = q_0^2 + \vec{q} \cdot \vec{q} = 1$

q_0 is called the **real part** of Q and $-i(q_1 \sigma_1 + q_2 \sigma_2 + q_3 \sigma_3) \equiv -i \vec{q} \cdot \vec{\sigma}$ is called the **imaginary part** or the **vector part**.

- $\vec{q} \cdot \vec{\sigma}$ alone, without $-i$, is a traceless, Hermitian 3-vector representation. This comes up as a way to recover physically measurable angular velocity from the quaternion.
- “Faithfully” means that the entire Lie Group $SU(2)$ can be smoothly covered by the unit quaternions as a coordinate chart with no singularities. Any abstract “point” in the manifold $SU(2)$ is uniquely specified by a unit quaternion Q and vice versa.
- The unit-norm constraint $(q_0^2 + q_1^2 + q_2^2 + q_3^2 = 1)$ means that the four coefficients are confined to the 3-sphere, $S^3 \subset \mathbb{R}^4$, just as in 3 dimensions, where the constraint $(x^2 + y^2 + z^2 = 1)$ confines (x, y, z) to the 2-sphere, $S^2 \subset \mathbb{R}^3$.
- We identify elements of $SU(2)$ with their quaternion coordinate representations, writing Q for either.

The Vector Part of a Quaternion is in $\mathfrak{su}(2)$

The vector part, $-i \vec{q} \cdot \vec{\sigma}$, is a member of $\mathfrak{su}(2)$ because the $\vec{q}$ are real.

The basis of $\mathfrak{su}(2)$ Models Hamilton’s Imaginary Units

The following proofs show that the $(-i \vec{\sigma})$ model Hamilton’s imaginary quaternion units $\{\mathbf{i}, \mathbf{j}, \mathbf{k}\}$, where $(\mathbf{i}^2 = \mathbf{j}^2 = \mathbf{k}^2 = -1)$ (they are imaginary), $\mathbf{i} \mathbf{j} = \mathbf{k}$, $\mathbf{j} \mathbf{k} = \mathbf{i}$, $\mathbf{k} \mathbf{i} = \mathbf{j}$ (cyclic property), and $(\mathbf{i} \mathbf{j} = -\mathbf{j} \mathbf{i})$, etc. (anti-commutativity).

$-i \vec{\sigma}$ is Imaginary

Every $-i \sigma_k$ squares to $-\mathbb{I}_2$, modeling Hamilton’s imaginary units.

```
In[61]:= proof@Echo[
  And @@ Table[(-i σ[k]).(-i σ[k]) == -id, {k, 3}],
  "-i σk squares to -I2:"];
```

» $-i \sigma_k$ squares to $-\mathbb{I}_2$: True

QED ■

$-i\vec{\sigma}$ is Cyclic

```
In[62]:= proof[
  (-i σ[1]) . (-i σ[2]) === -i σ[3] &&
  (-i σ[2]) . (-i σ[3]) === -i σ[1] &&
  (-i σ[3]) . (-i σ[1]) === -i σ[2];
```

QED ■

Concisely

```
In[63]:= proof@Echo[
  And @@ Table[
    With[{i = RotateLeft[{1, 2, 3}, k]},
      (-i σ[i[[1]]) . (-i σ[i[[2]]) === (-i σ[i[[3]])],
      {k, 0, 2}],
    "-iσ is cyclic :"];
```

» $-i\vec{\sigma}$ is cyclic : True

QED ■

$+i\vec{\sigma}$ is Not Cyclic (Ancillary Fact)

Note that $(+i\sigma_k)$ would not do:

```
In[64]:= proof@Echo[
  Not[Or @@ Table[
    With[{i = RotateLeft[{1, 2, 3}, k]},
      (+i σ[i[[1]]) . (+i σ[i[[2]]) === (+i σ[i[[3]])],
      {k, 0, 2}]],
    "+iσ is not cyclic :"];
```

» $+i\vec{\sigma}$ is not cyclic : True

QED ■

$-i\vec{\sigma}$ is Anti-Commutative

Finally, the members of $-i\vec{\sigma}$ are anti-commutative, just like Hamilton's imaginary units.

```
In[65]:= proof@Echo[
  And @@ And @@@ Table[
    (-i σ[i]) . (-i σ[j]) === -(-i σ[j]) . (-i σ[i]),
    {i, 3},
    {j, i + 1, 3}],
  "-iσ is anti-commutative :"];
```

» $-i\vec{\sigma}$ is anti-commutative : True

QED ■

∴ Pure Imaginary Quaternions are Isomorphic to $\mathfrak{su}(2)$

The significance of the fact that the vector or imaginary part of a quaternion is in $\mathfrak{su}(2)$ is both geometrical and physical, with two major implications:

1. Geometrical: The Lie Group $SU(2)$ is the 3-sphere $S^3 \subset \mathbb{R}^4$. The identity element $\mathbb{I}_2$ is the point $(1, 0, 0, 0)$ —the "North Pole" of that sphere, specifying "no rotation." The tangent space to the 3-sphere at the North Pole is the flat, 3D hyperplane perpendicular to the q_0 axis, consisting of all points $(0, q_1, q_2, q_3)$. In this hyperplane, the points need not square up to unity.
2. Physical: In classical mechanics, the rotator of a body is a unit quaternion Q in the group $SU(2)$. But the angular velocity of the rotating body ω is $\vec{\omega}$, a vector represented in the algebra $\mathfrak{su}(2)$, whose norm is not necessarily 1.

SUMMARY: $\mathfrak{su}(2)$ is the vector space of pure quaternions. While $SU(2)$ describes the orientation of a system, its Lie algebra, $\mathfrak{su}(2)$, describes the axis and rate of the system's rotation. The $-i$ factor in $-i\vec{\omega} \cdot \vec{\sigma}$ ensures that the "imaginary" units of the matrix representation correctly mirror the anti-commuting units $\{\mathbf{i}, \mathbf{j}, \mathbf{k}\}$ of Hamilton's quaternions.

Generators of Rotations

The real part, q_0 , of the quaternion Q multiplies the identity matrix. The imaginary or vector part, $\vec{q} \equiv (q_1, q_2, q_3)$, which represents angular velocity, is constructed from a real 3-vector, $\vec{q} = \vec{\omega} \equiv (\omega_1, \omega_2, \omega_3)$, that points along the instantaneous "twist axis." As we shall see immediately below, q_0 is the cosine of half the twist *angle*, and the imaginary part is $-i$ times the sine of half the twist angle times the normalized $\vec{\omega}$ expanded in $\mathfrak{su}(2)$, $-i(\vec{\omega} \cdot \vec{\sigma} / |\vec{\omega}|)$. Because $\cos^2 + \sin^2 = 1$, the entire quaternion is of unit magnitude even when $\vec{q}$ is not of unit magnitude. In fact, if $|\vec{q}| = 0$, the twist vector vanishes, there is no rotation, and the entire quaternion is $(q_0 = 1) \mathbb{I}_2$.

Exponential Map $\mathfrak{su}(2) \rightarrow SU(2)$, Angle to Quaternion

In mechanics, classical or quantum, we usually specify an instantaneous angle of rotation by $t\vec{\omega}$, a coordinate time t multiplied by an angular velocity 3-vector $\vec{\omega}$. Its representation in $\mathfrak{su}(2)$ is $-i t \vec{\omega} \cdot \vec{\sigma}$:

```
In[66]:= proof@Module[{ω = {ω1, ω2, ω3}, θRep},
  Echo[ {θRep = -i t ω.σVec // ComplexExpand} // MatrixForm,
    "instantaneous angle is in su(2) :"];
  AntiHermitianQ[θRep] && TracelessQ[θRep];
```

» instantaneous angle is in $\mathfrak{su}(2)$: $\begin{pmatrix} -i t \omega_3 & -i t \omega_1 - t \omega_2 \\ -i t \omega_1 + t \omega_2 & i t \omega_3 \end{pmatrix}$

QED ■

We map this angle representation in $\mathfrak{su}(2)$ into a normalized orientation quaternion (versor) $Q(t)$ in

$SU(2)$ via a matrix exponential:

$$\begin{aligned} Q(t) &= e^{-i\frac{\omega t}{2}\left(\frac{\vec{\omega} \cdot \vec{\sigma}}{\omega}\right)} = e^{-it\left(\frac{\vec{\omega} \cdot \vec{\sigma}}{2}\right)} = \cos\left(\frac{\omega t}{2}\right) \mathbb{I}_2 - i \sin\left(\frac{\omega t}{2}\right) \left(\frac{\vec{\omega} \cdot \vec{\sigma}}{\omega}\right) \\ &= \cos\left(\frac{\omega t}{2}\right) \mathbb{I}_2 - i \sin\left(\frac{\omega t}{2}\right) \left(\frac{1}{\omega} \begin{pmatrix} \omega_3 & \omega_1 - i\omega_2 \\ \omega_1 + i\omega_2 & -\omega_3 \end{pmatrix}\right) \end{aligned} \quad (6)$$

Here is a proof that the `MatrixExp` equals the longhand form:

```
In[67]:= proof@Module[{ω, ωVec = {ω1, ω2, ω3}, lieGroupElement, expOfLieAlgebraElement},
  lieGroupElement =
    FullSimplify[
      Cos[ $\frac{\omega t}{2}$ ] id - i Sin[ $\frac{\omega t}{2}$ ] ( $\frac{\omega \text{Vec} \cdot \sigma \text{Vec}}{\omega}$ )];
  expOfLieAlgebraElement =
    FullSimplify[
      MatrixExp[-i t ( $\frac{\omega \text{Vec} \cdot \sigma \text{Vec}}{2}$ )] // .
      {ωVec.ωVec → ω2}]];
  lieGroupElement === expOfLieAlgebraElement];
```

QED ■

Example

Let $\vec{\omega} = \{0, 0, \theta\}$ be a rotation of angle θ around the Z axis and let $t = 1$. Then the following expression, $Q.r.Q^\dagger$, rotates the vector r counterclockwise

$$Q = \cos\left(\frac{\theta}{2}\right) \mathbb{I}_2 - i \sin\left(\frac{\theta}{2}\right) * \left(\frac{1}{\theta} \begin{pmatrix} \theta & 0 \\ 0 & -\theta \end{pmatrix}\right) = \cos\left(\frac{\theta}{2}\right) \mathbb{I}_2 - i \sin\left(\frac{\theta}{2}\right) \sigma_3 \quad (7)$$

```
In[68]:= With[{Q = Cos[ $\frac{\theta}{2}$ ] id - i Sin[ $\frac{\theta}{2}$ ] σ[3], r = {x, y, z}.σVec},
  Q.r.Q†
  // Simplify // ComplexExpand] // MatrixForm
```

```
Out[68]//MatrixForm=

$$\begin{pmatrix} z & x \cos[\theta] - y \sin[\theta] + i (-y \cos[\theta] - x \sin[\theta]) \\ x \cos[\theta] - y \sin[\theta] + i (y \cos[\theta] + x \sin[\theta]) & -z \end{pmatrix}$$

```

Spot the X component, $x \cos \theta - y \sin \theta$, of the new vector, as the real part of the $\llbracket 1, 2 \rrbracket$ or the $\llbracket 2, 1 \rrbracket$ component. Compare it with the intuited or memorized 2D rotation from school math to see that the new vector is, indeed, rotated by θ counterclockwise around the Z axis, specified by σ_3 .

Notice that the result of the rotation remains Hermitian and traceless for any real-vector state r , thus a valid representation of the rotated state:

```
In[69]:= proof@With[{Q = Cos[ $\frac{\theta}{2}$ ] id -  $\mathbf{i}$  Sin[ $\frac{\theta}{2}$ ]  $\sigma[3]$ , r = {x, y, z}. $\sigma$ Vec},
  {rPrime = Q.r.Q† // Simplify // ComplexExpand},
  HermitianQ[rPrime] && TracelessQ[rPrime]};
```

QED ■

Chapter III: Geodesic Equations of Motion in $SU(2)$

Here we solve directly for the geodesics of $SU(2)$ with kinetic energy as the metric tensor, in a manner mirroring the same derivation in $SO(3)$. These geodesics represent all trajectories of our baton as an inverted spherical pendulum (no spin) and as a symmetrical top (non-zero spin).

Geometric Approach

The thing to appreciate here is the brevity, regularity, and parallelism to our previous development in $SO(3)$. We have added some explanatory headlines and prose, plus TODO markers for future analysis, but the sketch is the same:

1. kinetic energy in the metric $\Rightarrow$
2. Christoffel symbols and the absolute derivative $\Rightarrow$
3. direct potential energy in contravariant form (outside the metric) $\Rightarrow$
4. equations of motion (eqns4) $\Rightarrow$
5. numerical solutions, manipulable plots and animations.

In $SO(3)$, the equations and solution were automatically constrained to the curved surface of the manifold via trigonometric functions. In $SU(2)$, we must explicitly constrain the equations and solutions to adhere to the manifold by insisting that $Q(t).Q(t)^\dagger = 1$, both algebraically, by static rules, and dynamically, via a Lagrange multiplier. This addition to the contravariant force term is the only deviation from the sketch of “pure geodesics on manifolds.”

```
In[70]:= ClearAll[coords4, eqns4, v4, r4, g4, ig4, I1, I3,  $\omega$ 0,  $\omega$ 1,  $\omega$ 2,  $\omega$ 3, t];
```

Coordinates

Begin by treating the $SU(2)$ group manifold (the 3-sphere S^3) as a 4D, non-Euclidean space, isomorphic to Hamilton’s quaternions with unit norm as shown in Chapter III.

```
In[71]:= coords4 = {q0, q1, q2, q3};
```

Angular Velocity in the 4D Basis

These are coefficients of $\dot{Q}$ in the Ω body-frame; proofs below (ω_0 is the radial direction, constrained to the hypersphere S^3):

```
In[72]:=  $\omega_1 = \{-q_1, q_0, q_3, -q_2\};$ 
 $\omega_2 = \{-q_2, -q_3, q_0, q_1\};$ 
 $\omega_3 = \{-q_3, q_2, -q_1, q_0\};$ 
 $\omega_0 = \{q_0, q_1, q_2, q_3\};$ 
```

Metric Tensor from Kinetic Energy

Because we use four real coordinates (q_0, q_1, q_2, q_3) to describe a real 3D manifold (Euclidean 3-space), the “natural” metric from kinetic energy is singular: does not account for radial motion. The toolkit’s `InverseMetric` and Christoffel functions require a “radial stabilizer” term in the metric. This doesn’t change the physics on the sphere but makes the geometry non-singular.

Add a ‘ghost’ inertia, `Iref`, for the radial direction; value determined by trial-and-error (TODO: derive by analysis). The metric is both complicated and surprisingly regular. Its validity is demonstrated by the numerical results rather than by analysis. Analysis is omitted, here, for brevity, though analysis is a worthy objective (TODO).

```
In[76]:= With[{Iref = 1/2},
Echo[metric4 =
4 (I1 Outer[Times,  $\omega_1$ ,  $\omega_1$ ] +
I1 Outer[Times,  $\omega_2$ ,  $\omega_2$ ] +
I3 Outer[Times,  $\omega_3$ ,  $\omega_3$ ] +
Iref Outer[Times,  $\omega_0$ ,  $\omega_0$ ])] // MatrixForm, "metric 4 :"];
» metric 4 : 
$$\begin{pmatrix} 4 \left( \frac{q_0^2}{2} + I_1 q_1^2 + I_1 q_2^2 + I_3 q_3^2 \right) & 4 \left( \frac{q_0 q_1}{2} - I_1 q_0 q_1 + I_1 q_2 q_3 - I_3 q_2 q_3 \right) & 4 \left( \frac{q_0 q_2}{2} - I_1 q_0 q_2 + I_1 q_1 q_3 - I_3 q_1 q_3 \right) & 4 \left( \frac{q_0 q_3}{2} - I_3 q_0 q_3 \right) \\ 4 \left( \frac{q_0 q_1}{2} - I_1 q_0 q_1 + I_1 q_2 q_3 - I_3 q_2 q_3 \right) & 4 \left( I_1 q_0^2 + \frac{q_1^2}{2} + I_3 q_2^2 + I_1 q_3^2 \right) & 4 \left( I_1 q_0 q_2 + I_1 q_1 q_3 - I_3 q_2 q_3 \right) & 4 \left( -I_1 q_0 q_2 + I_3 q_0 q_2 + \frac{q_1 q_3}{2} - I_1 q_1 q_3 \right) \\ 4 \left( \frac{q_0 q_2}{2} - I_1 q_0 q_2 + I_1 q_1 q_3 - I_3 q_1 q_3 \right) & 4 \left( I_1 q_0 q_2 + I_1 q_1 q_3 - I_3 q_2 q_3 \right) & 4 \left( I_1 q_0^2 + \frac{q_1^2}{2} + I_3 q_2^2 + I_1 q_3^2 \right) & 4 \left( I_1 q_0 q_1 + I_1 q_2 q_3 - I_3 q_2 q_3 \right) \\ 4 \left( \frac{q_0 q_3}{2} - I_3 q_0 q_3 \right) & 4 \left( -I_1 q_0 q_2 + I_3 q_0 q_2 + \frac{q_1 q_3}{2} - I_1 q_1 q_3 \right) & 4 \left( I_1 q_0 q_1 + I_1 q_2 q_3 - I_3 q_2 q_3 \right) & 4 \left( I_1 q_0^2 + \frac{q_1^2}{2} + I_3 q_2^2 + I_1 q_3^2 \right) \end{pmatrix}$$

```

```
In[77]:= (invMetric4 = (Inverse[metric4] // FullSimplify));
```

Christoffel Connection with Quaternion Normalization

```
In[78]:= g4 = MakeIndexer[metric4, coords4];
ig4 = MakeIndexer[invMetric4, coords4];
(* manually constructed indexer, just as with S0(3) *)
r4[ $\mu$ _,  $\nu$ _,  $\lambda$ _] := ChristoffelsFromMetricIndexer[g4, ig4,  $\mu$ ,  $\nu$ ,  $\lambda$ , coords4];
(* add general normalization  $Q \cdot Q^* \rightarrow 1$ ; time dependence injected later *)
r4red = MakeIndexer[
Table[r4[ $\mu$ ,  $\nu$ ,  $\lambda$ ], { $\mu$ , coords4}, { $\nu$ , coords4}, { $\lambda$ , coords4}] //.
{ $q_0^2 + q_1^2 + q_2^2 + q_3^2 \rightarrow 1$ }, coords4];
```

Absolute Derivative Term

While the connection does not have the explicit time dependence $[t]$ needed for numerical work, `AbsoluteDerivativeFromConnectionIndexer` internally injects such time-dependence. Time injection at this phase is also a feature of our $SO(3)$ development:

```
In[82]:= v4 = MakeIndexer[(c ↦ c'[t]) /@ coords4, coords4];
ad4 =
  (AbsoluteDerivativeFromConnectionIndexer[v4, r4red, t, coords4] // FullSimplify);
```

Potential Energy in $SU(2)$

from the Z-projection of the baton; `PotentialScalarFieldFromMetricIndexer` also internally injects explicit time dependence, just as with $SO(3)$:

```
In[84]:= VSU2 = m g cz (q0^2 - q1^2 - q2^2 + q3^2);
(* add time-dependent normalization Q(t).Q(t)† → 1 *)
pf4 = ((PotentialScalarFieldFromMetricIndexer[VSU2, ig4, t, coords4] //.
  {q0[t]^2 + q1[t]^2 + q2[t]^2 + q3[t]^2 → 1}) // FullSimplify);
```

Contravariant Constraint Force Vector

Forces are usually covariant. However, we are leaving constraint forces out of the metric, just as we did for $SO(3)$, so that we can have Newtonian time. Because the absolute derivative is contravariant, and because we convert the potential field gradient into contravariant form by raising its indices via the contravariant inverse metric, we must do likewise with the Lagrange constraint force. Here is its contravariant form, again without analysis for brevity (TODO).

```
In[86]:= lagrangeMultiplier = λ[t];
(* cn4 is the contravariant constraint force vector *)
cn4 = lagrangeMultiplier * {q0[t], q1[t], q2[t], q3[t]};
```

Geometrical Equations of Motion

The equations of motion are displayed here for later comparison to a Lagrangian approach in $SU(2)$:

```
In[88]:= (eqns4 = (MapThread[Equal, {ad4, pf4 + cn4}] // FullSimplify)) // ColumnForm
Echo[Length[eqns4], "number of geometrical equations of motion :"];
Out[88]=
```

$$\frac{2 q_2[t] \left((I_1 q_2[t]^2 + (2 I_1 - I_3) q_3[t]^2) q_0'[t] + (-I_1 + I_3) q_2[t] \times q_3[t] q_1'[t] \right) q_2'[t] + 2 \left(q_3[t] \left(I_3 q_2[t]^2 + I_1 q_3[t]^2 \right) q_0'[t] + (I_1 - I_3) q_1[t] \left(I_1 q_2[t]^2 + (2 I_1 - I_3) q_3[t]^2 \right) q_0'[t] + q_2[t] \left(I_1 q_2[t]^2 + I_3 q_3[t]^2 \right) q_1'[t] \right) q_2'[t] + 2 q_3[t] \left((-I_1 + I_3) q_2[t] \times q_3[t] q_0'[t] + (2 I_1 - I_3) q_2[t]^2 + I_1 q_3[t]^2 \right) q_0'[t] + (-I_1 + I_3) q_2[t] \times q_3[t] q_1'[t]}{I_1}$$

$$0. - 9.81 \text{ cz m } q_2[t]^3 + 9.81 \text{ cz m } q_2[t] q_3[t]^2 - \frac{9.81 \text{ cz m } q_2[t] q_3[t]^2}{I_1} - 1. q_2[t] \times \lambda[t] + 1. q_2[t]^3 q_0'$$

$$0. - 9.81 \text{ cz m } q_2[t]^2 q_3[t] + \frac{9.81 \text{ cz m } q_2[t]^2 q_3[t]}{I_1} + 9.81 \text{ cz m } q_3[t]^3 - 1. q_3[t] \times \lambda[t] + 3. q_2[t]^2 q_3[t]$$

» number of geometrical equations of motion : 4


```

q2[0] == I / 2 Tr[Q0Matrix .  $\sigma$ [2]],
q3[0] == I / 2 Tr[Q0Matrix .  $\sigma$ [3]],
q0'[0] == 1 / 2 Tr[dQ0Matrix],
q1'[0] == I / 2 Tr[dQ0Matrix .  $\sigma$ [1]],
q2'[0] == I / 2 Tr[dQ0Matrix .  $\sigma$ [2]],
q3'[0] == I / 2 Tr[dQ0Matrix .  $\sigma$ [3]]];
(*NDSolve the ODEs*)
soln = NDSolve[Flatten[{
  q0'[t] == (q0'[t] /. odes),
  q1'[t] == (q1'[t] /. odes),
  q2'[t] == (q2'[t] /. odes),
  q3'[t] == (q3'[t] /. odes), ics}] /.
  (*substitute hard-
    coded value for I1 in I3 so that Replace can reduce in one shot*)
  {m → 1, cz → 1.2, I1 → 1.22, I3 → rI31 * 1.22},
  {q0[t], q1[t], q2[t], q3[t], q0'[t], q1'[t], q2'[t], q3'[t]},
  {t, 0, tLim}];
(*Extract the 8 interpolating functions*)
q0x = soln[[1, 1, 2]]; Dq0x = soln[[1, 5, 2]];
q1x = soln[[1, 2, 2]]; Dq1x = soln[[1, 6, 2]];
q2x = soln[[1, 3, 2]]; Dq2x = soln[[1, 7, 2]];
q3x = soln[[1, 4, 2]]; Dq3x = soln[[1, 8, 2]];
flt = " $(\delta_0)$ " == ToString[ $\delta_0$ ] <>
  " $^\circ$ ,  $(\eta_0)$ " == ToString[ $\eta_0$ ] <> " $^\circ$ ",  $\psi_0'$  == ToString[D $\psi_0$ ];
GraphicsGrid[{
  {Plot[q0x, {t, 0, tL},
    FrameLabel → {{"(q0)", ""}, {"t [sec]", flt}}, Frame → True],
    Plot[Dq0x, {t, 0, tL},
    FrameLabel → {{"(q̇0)", ""}, {"t [sec]", flt}}, Frame → True]},
  {Plot[q1x, {t, 0, tL},
    FrameLabel → {{"(q1)", ""}, {"t [sec]", flt}}, Frame → True],
    Plot[Dq1x, {t, 0, tL},
    FrameLabel → {{"(q̇1)", ""}, {"t [sec]", flt}}, Frame → True]},
  {Plot[q2x, {t, 0, tL},
    FrameLabel → {{"(q2)", ""}, {"t [sec]", flt}}, Frame → True],
    Plot[Dq2x, {t, 0, tL},
    FrameLabel → {{"(q̇2)", ""}, {"t [sec]", flt}}, Frame → True]},
  {Plot[q3x, {t, 0, tL},
    FrameLabel → {{"(q3)", ""}, {"t [sec]", flt}}, Frame → True],

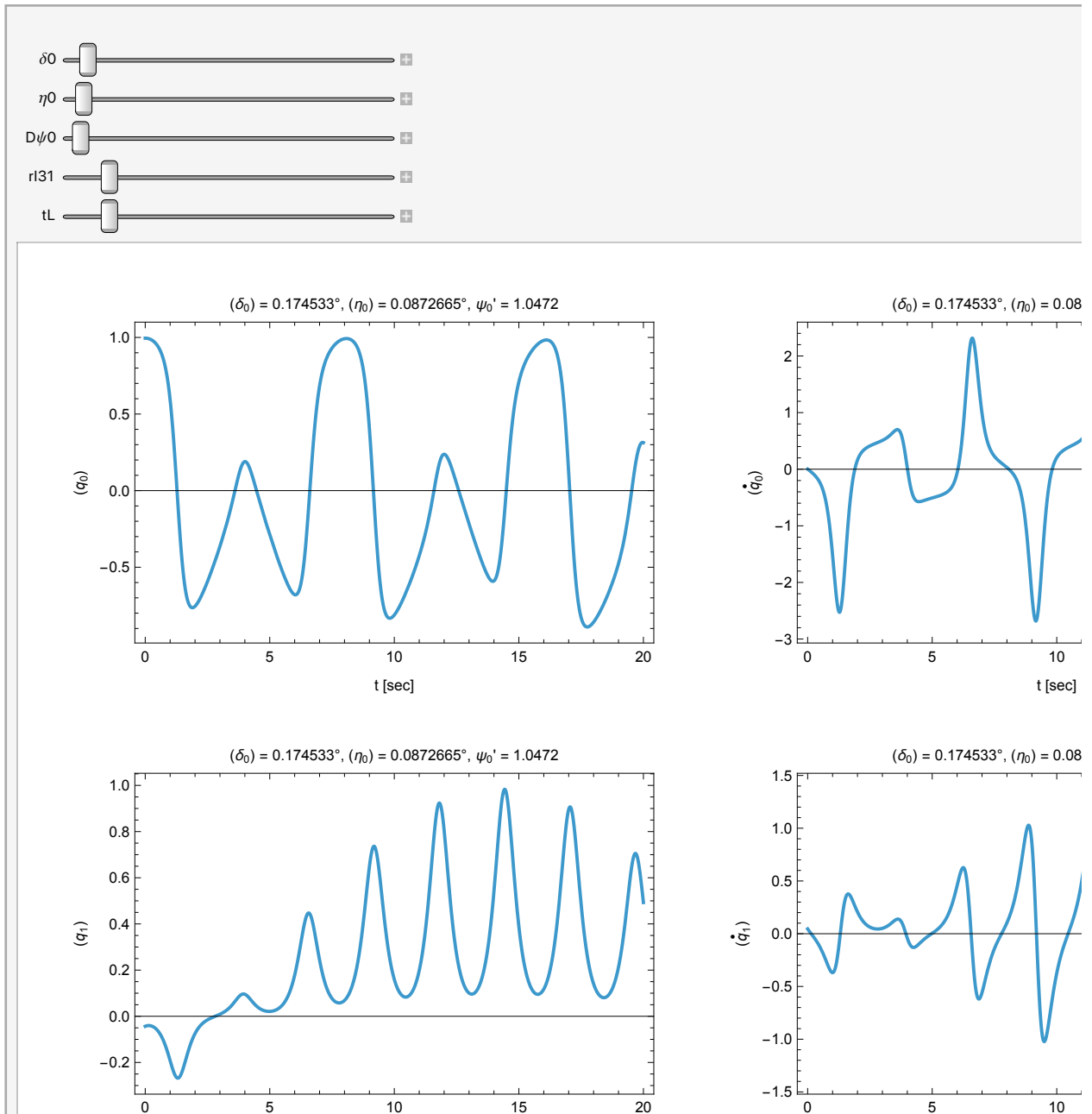
```

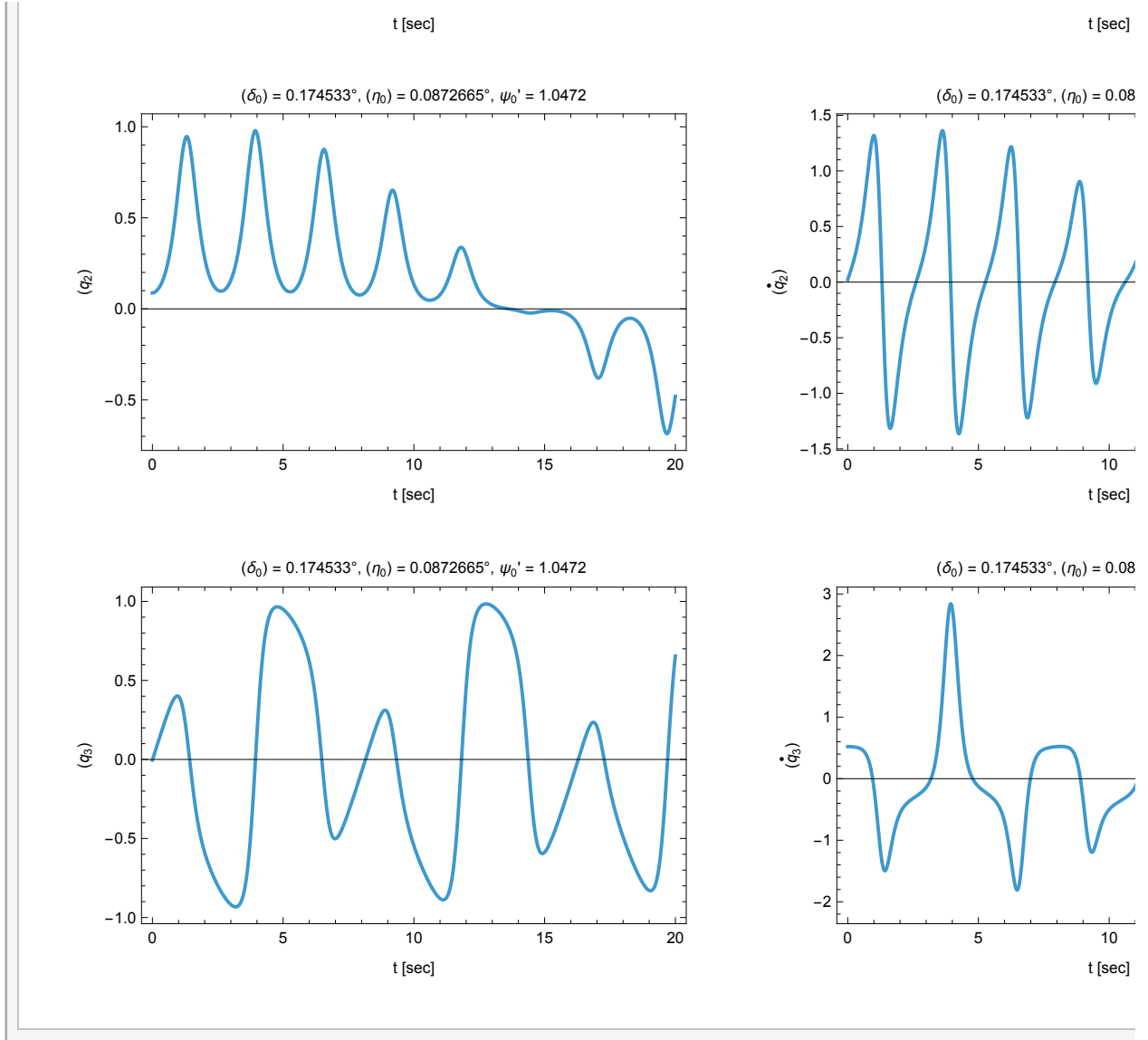
```

Plot[Dq3x, {t, 0, tL},
  FrameLabel → {{{"(q̇₃)", ""}, {"t [sec]", flt}}, Frame → True}],
ImageSize → 800]],
{{δ0, 10. °}, 0., 360. °},
{{η0, 5. °}, 0, 360. °},
{{Dψ0, 10 // radPerSec}, 0, 300.},
{{rI31, 1}, 0, 10},
{{tL, tLim/10}, 0, tLim}, SaveDefinitions → True];
interactivePlotsOfODEs[pureODEsE4]

```

Out[95]=





Derivation of Angular Velocity in the 4D Basis

This is a derivation of the particular convention we need for physics: the body frame. It justifies the definition of ω above.

IMPORTANT: In Chapter II, we defined $\mathfrak{su}(2)$ in the space frame, defining angular velocity, $\vec{\omega}_{\text{SPACE}}$, with respect to the right-invariant group transport $\dot{Q} \cdot Q^\dagger$. That is the standard starting point for proving that the algebra $\mathfrak{su}(2)$ consists of traceless, anti-Hermitian matrices. However, for dynamics of the pendulum or top, we must switch to the body frame, defining angular velocity, $\vec{\omega}_{\text{BODY}}$, with respect to the left-invariant group transport, $Q^\dagger \cdot \dot{Q}$, because the moments of inertia I_1 and I_3 are constant only in this co-rotating frame. Only the body frame ensures that the metric remains constant and the equations of motion describe pure geodesics.

Start from the fundamental relationship between the quaternion trajectory $Q(t)$ and the body-fixed angular velocity matrix Ω :

$$\Omega \equiv \vec{\omega} \cdot \vec{\sigma} = 2i Q^\dagger \dot{Q} \quad (\text{traceless, Hermitian}) \quad (8)$$

which follows from the exponential map

$$\begin{aligned} Q(t) &= e^{-it \left(\frac{\vec{\omega} \cdot \vec{\sigma}}{2} \right)} \\ \dot{Q}(t) &= Q(t) \left(-i \frac{\vec{\omega} \cdot \vec{\sigma}}{2} \right) \\ Q(t)^\dagger &= e^{it \left(\frac{\vec{\omega} \cdot \vec{\sigma}}{2} \right)} \\ Q(t)^\dagger \dot{Q}(t) &= -i \left(\frac{\vec{\omega} \cdot \vec{\sigma}}{2} \right) \quad (\text{in } \mathfrak{su}(2)) \end{aligned} \quad (9)$$

$$\vec{\omega} \cdot \vec{\sigma} = 2i Q^\dagger \dot{Q} \quad (\text{traceless, Hermitian})$$

- Note that the space-frame form, $\dot{Q}(t) = \left(-i \frac{\vec{\omega} \cdot \vec{\sigma}}{2} \right) Q(t)$, with $\left(-i \frac{\vec{\omega} \cdot \vec{\sigma}}{2} \right)$ on the left, is also true because $\left(-i \frac{\vec{\omega} \cdot \vec{\sigma}}{2} \right)$ is constant and $(A e^{At} = e^{At} A)$ for any constant matrix A . We choose $\left(-i \frac{\vec{\omega} \cdot \vec{\sigma}}{2} \right)$ on the right, in this section, to support derivations in the body frame.

Given $Q(t) = q_0 \mathbb{I}_2 - i \vec{q}(t) \cdot \vec{\sigma}$ and its time derivative $\dot{Q}(t) = \dot{q}_0 \mathbb{I}_2 - i \dot{\vec{q}}(t) \cdot \vec{\sigma}$, via Pauli cyclicity and anti-commutativity

$$\sigma_j \sigma_k = \delta_{jk} \mathbb{I}_2 + i \sum_{l=1}^3 \epsilon_{jkl} \sigma_l \quad (10)$$

where ϵ_{jkl} is the fully anti-symmetric Levi-Civita symbol, equaling 1 when jkl is an even permutation of (1, 2, 3), equaling -1 when jkl is an odd permutation, and equaling 0 otherwise, expand $2i Q^\dagger \dot{Q}$ (omitted).

By substituting these into the expression for Ω and collecting terms proportional to each σ_k , find that each component ω_k of $\vec{\omega}$ is a linear combination of the coordinate velocities $\dot{q}_{\mu \in \{0,1,2,3\}}$,

$$\omega_k = 2 \sum_{\mu=0}^3 (v_k)_\mu \dot{q}_\mu \quad (11)$$

where the three v_k are the coefficients we see in the derivations above and below. The choice of the body frame (left-invariant) flips the sign of the cross-product term in the Levi-Civita expansion, exactly matching the components v_k derived for the equations of motion. This is easier to see in Wolfram notation:

Setup

```
In[96]:= ClearAll[q, qdot, Q, Qd, QDag, Ω,
v1, v2, v3, v0,
q0, q1, q2, q3,
qd0, qd1, qd2, qd3];
```

Define Q and its Derivative in Terms of 4D Coordinates

```
In[97]:= q = {q0, q1, q2, q3};
         qdot = {dq0, dq1, dq2, dq3};

In[99]:= Q = q0 id - i (q[[2 ;; 4]] . σVec);
         Qd = dq0 id - i (qdot[[2 ;; 4]] . σVec);
         QDag = q0 id + i (q[[2 ;; 4]] . σVec);
```

Hermitian angular Velocity Matrix

This matrix is in the body frame, $Q^\dagger \dot{Q}$ instead of $\dot{Q} Q^\dagger$.

```
In[102]:= Ω = 2 i (QDag . Qd) // ComplexExpand // FullSimplify;
```

ω Components via Pauli Trace Property

$$\omega_k = \frac{1}{2} \text{Tr} \Omega \cdot \sigma_k \quad (12)$$

```
In[103]:= ωComponents = Table[1/2 Tr[Ω . σVec[[k]]], {k, 3}] // FullSimplify;
```

Coefficients v_k from Gradient w.r.t $\dot{Q}$

Matching the vectors defined in the geometric metric:

```
In[104]:= Echo[derivedV1 = D[ωComponents[[1]], {qdot}]/2, "derived v1 :"];
          (*Factor of 2 for normalization*)
          Echo[derivedV2 = D[ωComponents[[2]], {qdot}]/2, "derived v2 :"];
          Echo[derivedV3 = D[ωComponents[[3]], {qdot}]/2, "derived v3 :"];

» derived v1 : {-q1, q0, q3, -q2}
» derived v2 : {-q2, -q3, q0, q1}
» derived v3 : {-q3, q2, -q1, q0}
```

v_0 is the radial direction (gradient of the sphere constraint).

```
In[107]:= Echo[derivedV0 = D[q . q, {q}]/2, "derived v0 :"];;

» derived v0 : {q0, q1, q2, q3}
```

```
Out[107]= {q0, q1, q2, q3} ;; All
```

Formal Verification

In[108]:=

```
proof[derivedV1 === {-q1, q0, q3, -q2}];
proof[derivedV2 === {-q2, -q3, q0, q1}];
proof[derivedV3 === {-q3, q2, -q1, q0}];
proof[derivedV0 === {q0, q1, q2, q3}];
```

QED ■

QED ■

QED ■

QED ■

Side-by-Side Interactive Animation in $SO(3)$ and $SU(2)$

Aside from negligible differences in energy and angular momentum, the solutions appear identical for all settings of the sliders, with one exception. **Slide the δ_0 slider all the way to the right— $\delta_0 = 90^\circ$,** and the $SO(3)$ solution fails completely with a $\text{Sec}[\delta] = \text{ComplexInfinity}$, while the $SU(2)$ solution proceeds normally. **This is the main result of this notebook: no poles in $SU(2)$!**

In[112]:=

```
interactiveSideBySideAnimation[su2ODEs_] :=
  With[{epsilon = 0.0001, h = 1/15., w = 1/4., vu = 3.},
    {ztxt = -1.5, txt = 0, ytxt = -2, znl = 0.3},
    {kart = Cuboid[{-w, -w, epsilon}, {w, w, epsilon + h}],
     floor = Polygon[{{-vu, -vu, 0}, {vu, -vu, 0}, {vu, vu, 0}, {-vu, vu, 0}}],
     axisLabelStyle = text  $\mapsto$  Style[text, Red, Bold, 18, FontFamily  $\rightarrow$  "Courier New"],
     cb = rig["cb"], thin = {0, 0, .001}, s = 2, o = {0, 0, 0}},
    {c3 = Cylinder[{o, thin}, s cb[[3]]],
     b3 = Sphere[o, s cb[[3]]],
     t3x = Tube[{-s cb[[3]] e2, s cb[[3]] e2}, .01]}, Manipulate[
    DynamicModule[{
      (*SO(3) Variables*)
      ics3,  $\delta x$ ,  $\eta x$ ,  $\psi x$ ,  $D\delta x$ ,  $D\eta x$ ,  $D\psi x$ , soln3,
      (*SU(2) Variables*)
      Q $\eta$ , Q $\delta$ , Q0Matrix, OmegaBody0, dQ0Matrix, icsSU2,
      q0x, q1x, q2x, q3x, Dq0x, Dq1x, Dq2x, Dq3x, solnSU2,
      (*Shared Variables*)
      numRules},
    numRules = {m  $\rightarrow$  1, cz  $\rightarrow$  1.2, I1  $\rightarrow$  1.22, I3  $\rightarrow$  rI31 * 1.22};
    (* ===== *)
    (* Run SO(3) Solver *)
    (* ===== *)
    ics3 = { $\delta[0] = \delta_0$ ,  $\eta[0] = \eta_0$ ,  $\psi[0] = 0$ ,  $\delta'[0] = 0$ ,  $\eta'[0] = 0$ ,  $\psi'[0] = D\psi_0$ };
    soln3 = NDSolve[Append[eqns3, ics3] /. numRules,
```

```

    {δ[t], η[t], ψ[t], δ'[t], η'[t], ψ'[t]}, {t, 0, tLim}];
δx = soln3[[1, 1, 2]]; Dδx = soln3[[1, 4, 2]];
ηx = soln3[[1, 2, 2]]; Dηx = soln3[[1, 5, 2]];
ψx = soln3[[1, 3, 2]]; Dψx = soln3[[1, 6, 2]];
(* =====*)
(* Run SU(2) Solver *)
(* =====*)
Qη = Cos[η0 / 2] id - I Sin[η0 / 2] σ[1];
Qδ = Cos[δ0 / 2] id - I Sin[δ0 / 2] σ[2];
Q0Matrix = Qη.Qδ;
OmegaBody0 = Dψ0 * σ[3];
dQ0Matrix = -I / 2 * Q0Matrix.OmegaBody0;
icsSU2 = ComplexExpand[{
    q0[0] == 1 / 2 Tr[Q0Matrix],
    q1[0] == I / 2 Tr[Q0Matrix.σ[1]],
    q2[0] == I / 2 Tr[Q0Matrix.σ[2]],
    q3[0] == I / 2 Tr[Q0Matrix.σ[3]],
    q0'[0] == 1 / 2 Tr[dQ0Matrix],
    q1'[0] == I / 2 Tr[dQ0Matrix.σ[1]],
    q2'[0] == I / 2 Tr[dQ0Matrix.σ[2]],
    q3'[0] == I / 2 Tr[dQ0Matrix.σ[3]]}];
solnSU2 = NDSolve[Flatten[{
    q0'[t] == (q0'[t] /. su2ODEs),
    q1'[t] == (q1'[t] /. su2ODEs),
    q2'[t] == (q2'[t] /. su2ODEs),
    q3'[t] == (q3'[t] /. su2ODEs), icsSU2}] /. numRules,
    {q0, q1, q2, q3, q0', q1', q2', q3'},
    {t, 0, tLim}];
q0x = q0 /. solnSU2[[1]]; Dq0x = q0' /. solnSU2[[1]];
q1x = q1 /. solnSU2[[1]]; Dq1x = q1' /. solnSU2[[1]];
q2x = q2 /. solnSU2[[1]]; Dq2x = q2' /. solnSU2[[1]];
q3x = q3 /. solnSU2[[1]]; Dq3x = q3' /. solnSU2[[1]];
(* =====*)
(* Combined Animation *)
(* =====*)
Animate[
Module[{
    (*S03 Render Vars*)
    δ, η, ψ, Dδ, Dη, Dψ, csS03, renderPosS03,
    rotfnS03, pzS03, TS03, VS03, EnumS03, radiusS03, plotS03,
    (*SU2 Render Vars*)
    vq0, vq1, vq2, vq3, vDq0, vDq1, vDq2, vDq3, w1, w2, w3, TnumSU2, VnumSU2,
    EnumSU2, pzSU2, radiusSU2, rotMat, csSU2, renderPosSU2, rotfnSU2, plotSU2},

```

```

(*---Extract S0(3) Instantaneous State---*)
δ = δx /. t → u; η = ηx /. t → u; ψ = ψx /. t → u;
Dδ = Dδx /. t → u; Dη = Dηx /. t → u; Dψ = Dψx /. t → u;
csS03 = RotationMatrix[η, e1].RotationMatrix[δ, e2].cb;
renderPosS03 = -csS03[[1]] e1 - csS03[[2]] e2;
rotfnS03 = RotationTransform[η, e1]@*
  RotationTransform[δ, e2]@*RotationTransform[ψ, e3];
TS03 = 1/2 ((I1 /. numRules) * (Dδ2 + Dη2 * Cos[δ]2) +
  (I3 /. numRules) * (Dψ + Dη * Sin[δ])2);
VS03 = (m /. numRules) * 9.81 * (cz /. numRules) * Cos[δ] * Cos[η];
EnumS03 = TS03 + VS03;
pzS03 = ((I1 (Dδ Sin[η] - Dη Sin[δ] Cos[δ] Cos[η]) +
  I3 (Dψ + Dη Sin[δ]) Cos[δ] Cos[η]) /. numRules);
radiusS03 = Norm[{csS03[[1]], csS03[[2]]}];
(*---Build S0(3) Graphic---*)
plotS03 = Show[{
  Graphics3D[{
    Text[
      font["t = "<>nfm@u<>", E = "<>nfm@EnumS03<>", pz = "<>ToString@
        NumberForm[Chop[pzS03, 10-5], 6]], {txt, ytxt, ztxt - 2 znl}],
    Text[
      font["{δ0, η0} = "<>vfm@{δ0 / °, η0 / °}], {txt, ytxt, ztxt - 3 znl}],
    Text[font["{δ, η} = "<>vfm@{δ / °, η / °}], {txt, ytxt, ztxt - 4 znl}],
    {Black, t3x},
    {White, Opacity[.7 / 8],
      c3, b3, GeometricTransformation[c3, rotfnS03]},
    {Red, Sphere[csS03, 1 / 16.]},
    {Yellow, Opacity[.3 / 4], floor},
    {Green, Opacity[.6], Translate[kart, renderPosS03]},
    {White, Opacity[.75], Translate[
      Translate[
        GeometricTransformation[
          {Cylinder[{0, 0, -1.2`}, {0, 0, 1.2`}], 0.16`},
          If[Mod[Floor[10 u], 2] == 0, Red, Green],
          Thickness[0.01],
          Line[{0, 0, 1.225}, {0.8, 0, 1.225}]], rotfnS03], csS03],
        renderPosS03 + (epsilon + h) e3}], ImageSize → 450,
    Axes → True,
    AxesLabel → axisLabelStyle /@ {"X", "Y", "Z"},
    PlotRange → {{-vu, vu}, {-vu, vu}, {-vu, vu}},
    ParametricPlot3D[{radiusS03 Cos[ha], radiusS03 Sin[ha], csS03[[3]]},
      {ha, 0, 2 π}]]];
(*---Extract SU(2) Instantaneous State---*)

```

```

vq0 = q0x[u]; vq1 = q1x[u]; vq2 = q2x[u]; vq3 = q3x[u];
vDq0 = Dq0x[u]; vDq1 = Dq1x[u]; vDq2 = Dq2x[u]; vDq3 = Dq3x[u];
With[{norm = Sqrt[vq02 + vq12 + vq22 + vq32]},
  vq0 /= norm; vq1 /= norm; vq2 /= norm; vq3 /= norm;];
rotMat = {
  {vq02 + vq12 - vq22 - vq32,
   -2 vq0 vq3 + 2 vq1 vq2, 2 vq0 vq2 + 2 vq1 vq3},
  {2 vq0 vq3 + 2 vq1 vq2, vq02 - vq12 + vq22 - vq32,
   -2 vq0 vq1 + 2 vq2 vq3},
  {-2 vq0 vq2 + 2 vq1 vq3, 2 vq0 vq1 + 2 vq2 vq3,
   vq02 - vq12 - vq22 + vq32}};
csSU2 = rotMat.cb;
renderPosSU2 = -csSU2[[1]] e1 - csSU2[[2]] e2;
rotfnSU2 = AffineTransform[rotMat];
w1 = 2 (-vq1 vDq0 + vq0 vDq1 + vq3 vDq2 - vq2 vDq3);
w2 = 2 (-vq2 vDq0 - vq3 vDq1 + vq0 vDq2 + vq1 vDq3);
w3 = 2 (-vq3 vDq0 + vq2 vDq1 - vq1 vDq2 + vq0 vDq3);
TnumSU2 = 1/2 ((I1 /. numRules) * (w12 + w22) + (I3 /. numRules) * w32);
VnumSU2 = (m /. numRules) * 9.81 * (cz /. numRules) * rotMat[[3, 3]];
EnumSU2 = TnumSU2 + VnumSU2;
pzSU2 = (I1 /. numRules) * w1 * rotMat[[3, 1]] +
  (I1 /. numRules) * w2 * rotMat[[3, 2]] +
  (I3 /. numRules) * w3 * rotMat[[3, 3]];
radiusSU2 = Norm[{csSU2[[1]], csSU2[[2]]}];
(*---Build SU(2) Graphic---*)plotSU2 = Show[{
  Graphics3D[{
    Text[
      font["t = " <> nfm@u <> ", E = " <> nfm@EnumSU2 <> ", pz = " <> ToString@
        NumberForm[Chop[pzSU2, 10-6], 6]], {txt, ytxt, ztxt - 2 znl}],
    Text[
      font["{δ0, η0} = " <> vfm@{δ0 / °, η0 / °}], {txt, ytxt, ztxt - 3 znl}],
    Text[font["cs = " <> vfm@csSU2], {txt, ytxt, ztxt - 4 znl}],
    {Black, t3x},
    {White, Opacity[.7 / 8], c3, b3,
      GeometricTransformation[c3, AffineTransform[rotMat]]},
    {Red, Sphere[csSU2, 1 / 16.]},
    {Yellow, Opacity[.3 / 4], floor},
    {Green, Opacity[.6], Translate[kart, renderPosSU2]},
    {White, Opacity[.75],
      Translate[
        Translate[
          GeometricTransformation[

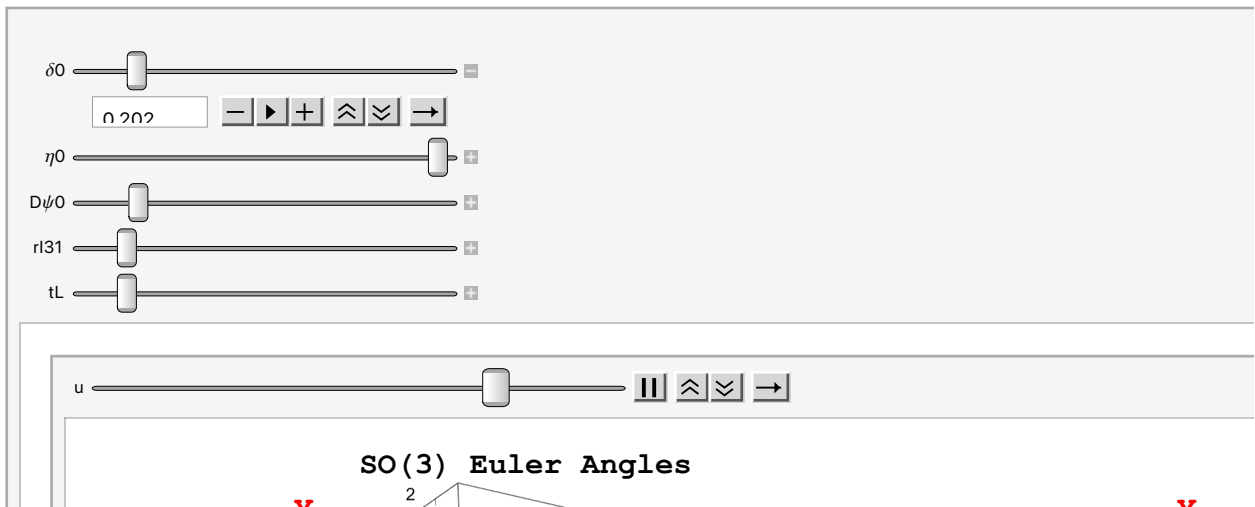
```

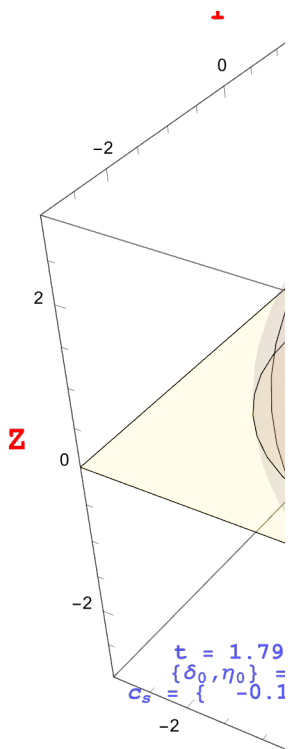
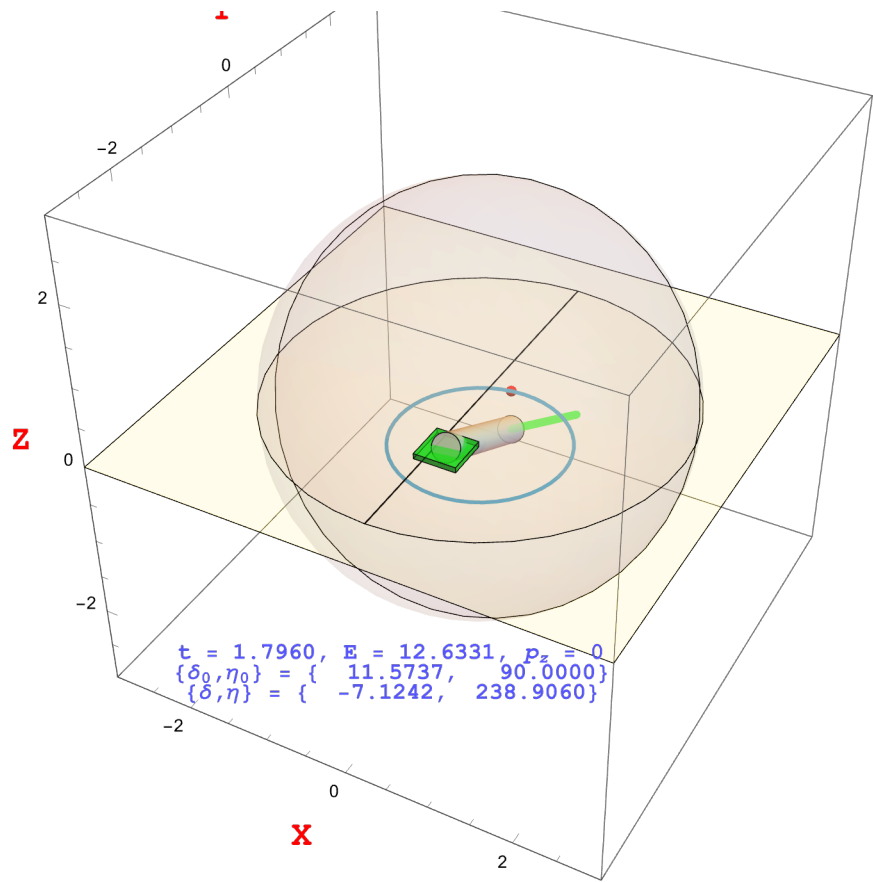
```

    {Cylinder[{{0, 0, -1.2}, {0, 0, 1.2}}, 0.16],
    If[Mod[Floor[10 u], 2] == 0, Red, Green],
    Thickness[0.01], (* little flag *)
    Line[{{0, 0, 1.225}, {0.8, 0, 1.225}}]],
    rotfnSU2],
    csSU2],
    renderPosSU2 + (epsilon + h) e3]]],
    ImageSize → 450,
    Axes → True,
    AxesLabel → axisLabelStyle /@ {"X", "Y", "Z"},
    PlotRange → {{-vu, vu}, {-vu, vu}, {-vu, vu}},
    ParametricPlot3D[
    {radiusSU2 Cos[ha], radiusSU2 Sin[ha], csSU2[[3]]}, {ha, 0, 2 π}]]];
(*---Render Side-by-Side---*)
Row[{
    Labeled[plotS03,
    Style["SO(3) Euler Angles", Bold, 16, FontFamily → "Courier New"],
    Top],
    Spacer[10],
    Labeled[plotSU2,
    Style["SU(2) Quaternions", Bold, 16, FontFamily → "Courier New"],
    Top]]],
    {u, 0, tL}, AnimationRate → .5]],
(*Controls*)
{{δ0, 10. °}, 0., 90. °},
{{η0, 5. °}, 0, 90. °},
{{Dψ0, 40 // radPerSec}, 0, 300 // radPerSec},
{{rI31, 1}, 0, 10},
{{tL, tLim / 10}, 0, tLim}, SaveDefinitions → True]];
interactiveSideBySideAnimation[pureODEsE4]

```

Out[113]=





Chapter IV: Alternative Lagrangian Approach in $SU(2)$

As a final verification, we produce an alternative derivation of the $SU(2)$ equations of motion via a traditional Lagrangian approach. It is one thing to appreciate that the Lagrangian approach, being an instance of the Principle of Least Action, must be equivalent to geodesics in $SU(2)$. It is another thing to see an interactive, numerical animation that demonstrates the fact. Although the equations of motion derived by the Lagrangian approach are different, in detail, from the equations of motion found by the pure geometric approach of geodesics, the fact that their numerical solutions are all but identical, drives home the point that we can derive the same results by two different, but mathematically equivalent, paths. We leave for future work, for brevity's sake, the analytical proof that the equations of motion by the two paths are equivalent, being satisfied, for now, with a numerical demonstration (TODO).

Euler-Lagrange Derivation in $SU(2)$

Setup

```
In[114]:=
ClearAll[q0, q1, q2, q3, qCoords, qVels, qIm, su2Eqns];
qCoords = {q0[t], q1[t], q2[t], q3[t]};
qVels = D[qCoords, t];

ClearAll[Q, qIm, QDag];
qIm[t_] := {q1[t], q2[t], q3[t]}; (* "Imaginary part" *)
Q[t_] := q0[t] id - i qIm[t] . sigmaVec;
QDag[t_] := (Q[t]† // ComplexExpand);
```

Potential Energy

```
In[121]:=
Rspace = Q[t] .  $\sigma$ [3] . QDag[t] // FullSimplify;
(*Z-component; projection onto Z-Pauli*)
zComponent = (1/2) Tr[Rspace .  $\sigma$ [3]] // ComplexExpand // FullSimplify;
V = m g c z zComponent;
```

Body Angular Velocity

```
In[124]:=
 $\Omega$ MatBody = 2 I QDag[t] . D[Q[t], t] // FullSimplify;
 $\omega$ [1] = (1/2) Tr[ $\Omega$ MatBody .  $\sigma$ [1]] // ComplexExpand // FullSimplify;
 $\omega$ [2] = (1/2) Tr[ $\Omega$ MatBody .  $\sigma$ [2]] // ComplexExpand // FullSimplify;
 $\omega$ [3] = (1/2) Tr[ $\Omega$ MatBody .  $\sigma$ [3]] // ComplexExpand // FullSimplify;
```

Kinetic Energy Plus Constraint Term

```
In[128]:=
T = (1/2) (I1  $\omega$ [1]2 + I1  $\omega$ [2]2 + I3  $\omega$ [3]2) // FullSimplify;
Lconstrained = (T - V) +  $\lambda$ [t] * (q0[t]2 + q1[t]2 + q2[t]2 + q3[t]2 - 1);
```

Euler-Lagrange Operator

```
In[130]:=
ClearAll[EL];
EL[L_, q_] := D[D[L, D[q, t]], t] - D[L, q] == 0;
```

Equations of Motion from Euler-Lagrange

In[132]:=

```
su2Eqns = Table[EL[Lconstrained, q] // FullSimplify, {q, qCoords}]
```

Out[132]=

```
{2. I1 q2[t] q0'[t] q2'[t] + 2. I3 q3[t] q0'[t] q3'[t] +
  2. I1 q2[t] q1'[t] q3'[t] + 1. I1 q1[t]^2 q0''[t] + 1. I1 q2[t]^2 q0''[t] +
  1. I3 q3[t]^2 q0''[t] + 1. I1 q2[t] × q3[t] q1''[t] + q1[t]
  (2. I1 q0'[t] q1'[t] + (-2. I1 + 2. I3) q2'[t] q3'[t] + (-1. I1 + 1. I3) q3[t] q2''[t]) +
  q0[t] (4.905 cz m - 0.5 λ[t] - 2. I3 q3'[t]^2 + I1
  (-2. q1'[t]^2 - 2. q2'[t]^2 - 1. q1[t] q1''[t] - 1. q2[t] q2''[t]) - 1. I3 q3[t] q3''[t]) =
  I3 q2[t] (2. q1'[t] q3'[t] + 1. q3[t] q1''[t]), 2. I1 q3[t] q0'[t] q2'[t] +
  2. I3 q2[t] q1'[t] q2'[t] + 2. I1 q3[t] q1'[t] q3'[t] + 1. I1 q2[t] × q3[t] q0''[t] +
  1. I1 q0[t]^2 q1''[t] + 1. I3 q2[t]^2 q1''[t] + 1. I1 q3[t]^2 q1''[t] + q0[t]
  (2. I1 q0'[t] q1'[t] + (-2. I1 + 2. I3) q2'[t] q3'[t] + (-1. I1 + 1. I3) q2[t] q3''[t]) +
  q1[t] (-4.905 cz m - 0.5 λ[t] - 2. I3 q2'[t]^2 - 1. I3 q2[t] q2''[t] +
  I1 (-2. q0'[t]^2 - 2. q3'[t]^2 - 1. q0[t] q0''[t] - 1. q3[t] q3''[t])) =
  I3 q3[t] (2. q0'[t] q2'[t] + 1. q2[t] q0''[t]),
  2. I1 q0[t] q0'[t] q2'[t] + 2. I3 q1[t] q1'[t] q2'[t] + 2. I1 q0[t] q1'[t] q3'[t] + q3[t]
  ((-2. I1 + 2. I3) q0'[t] q1'[t] + 2. I1 q2'[t] q3'[t] + (-1. I1 + 1. I3) q1[t] q0''[t]) +
  1. I1 q0[t]^2 q2''[t] + 1. I3 q1[t]^2 q2''[t] + 1. I1 q3[t]^2 q2''[t] +
  1. I1 q0[t] × q1[t] q3''[t] +
  q2[t] (-4.905 cz m - 0.5 λ[t] - 2. I3 q1'[t]^2 - 1. I3 q1[t] q1''[t] +
  I1 (-2. q0'[t]^2 - 2. q3'[t]^2 - 1. q0[t] q0''[t] - 1. q3[t] q3''[t])) =
  I3 q0[t] (2. q1'[t] q3'[t] + 1. q1[t] q3''[t]),
  2. I1 q1[t] q0'[t] q2'[t] + 2. I3 q0[t] q0'[t] q3'[t] + 2. I1 q1[t] q1'[t] q3'[t] + q2[t]
  ((-2. I1 + 2. I3) q0'[t] q1'[t] + 2. I1 q2'[t] q3'[t] + (-1. I1 + 1. I3) q0[t] q1''[t]) +
  1. I1 q0[t] × q1[t] q2''[t] + q3[t] (4.905 cz m - 0.5 λ[t] - 2. I3 q0'[t]^2 - 1. I3 q0[t]
  q0''[t] + I1 (-2. q1'[t]^2 - 2. q2'[t]^2 - 1. q1[t] q1''[t] - 1. q2[t] q2''[t])) +
  1. I3 q0[t]^2 q3''[t] + 1. I1 q1[t]^2 q3''[t] + 1. I1 q2[t]^2 q3''[t] =
  I3 q1[t] (2. q0'[t] q2'[t] + 1. q0[t] q2''[t]) }
```

(*Take the 2nd time derivative of the constraint*)

```
constraintEqn2 = D[q0[t]^2 + q1[t]^2 + q2[t]^2 + q3[t]^2 - 1, {t, 2}] == 0;
```

(*Solve for the highest derivatives and the constraint force*)

```
stateVars = {q0''[t], q1''[t], q2''[t], q3''[t], λ[t]};
```

```
pureODEsLA = Solve[Append[su2Eqns, constraintEqn2], stateVars][[1]];
```

Interactive Plots of the Euler-Lagrange Solution

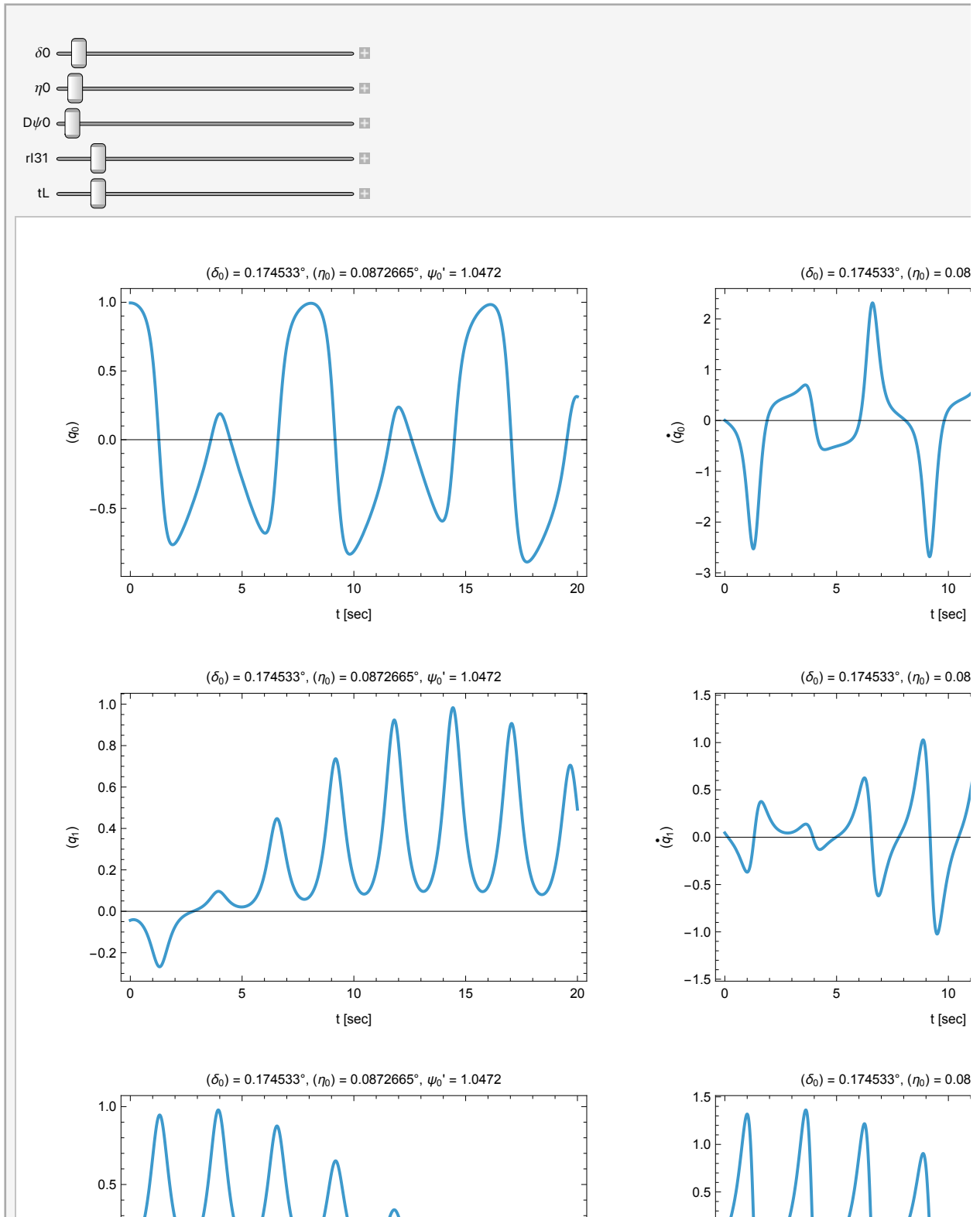
The thing to note here is that the plots are indistinguishable from the geometrical solution for any

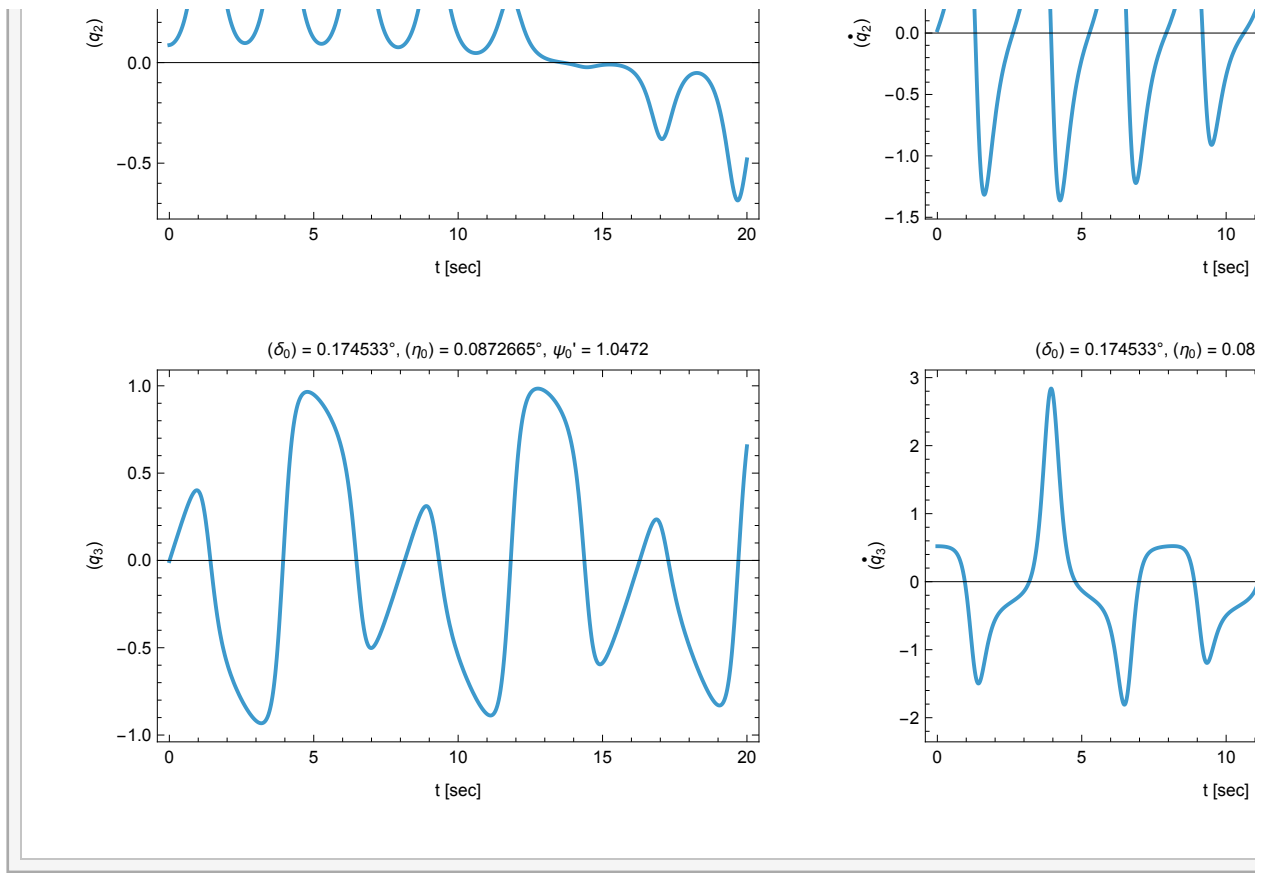
settings of the sliders.

In[136]:=

interactivePlotsOfODEs[pureODEsLA]

Out[136]=



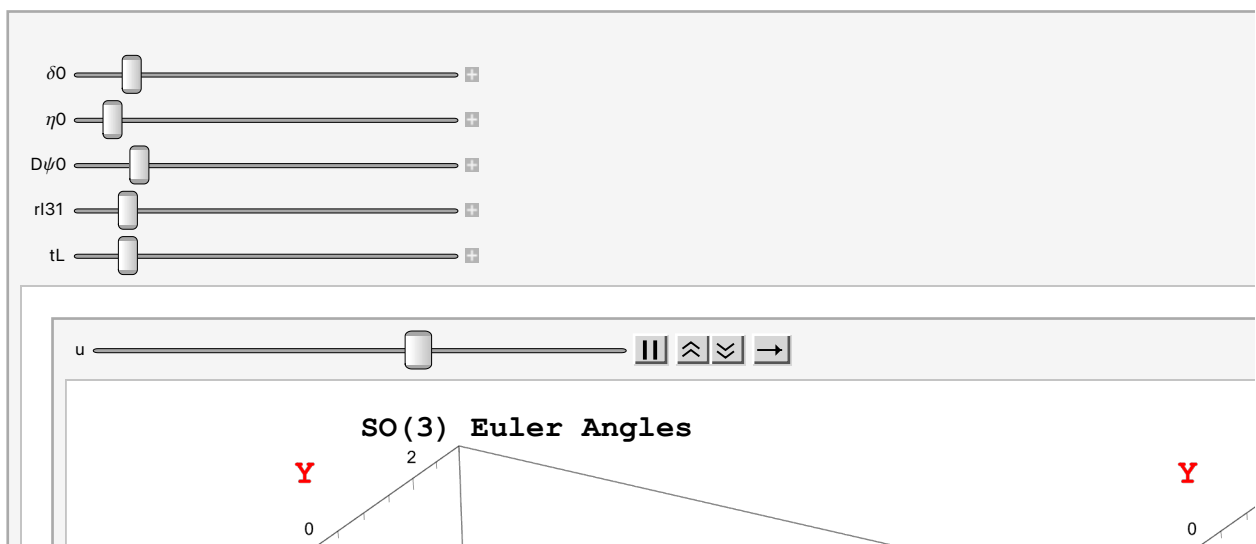


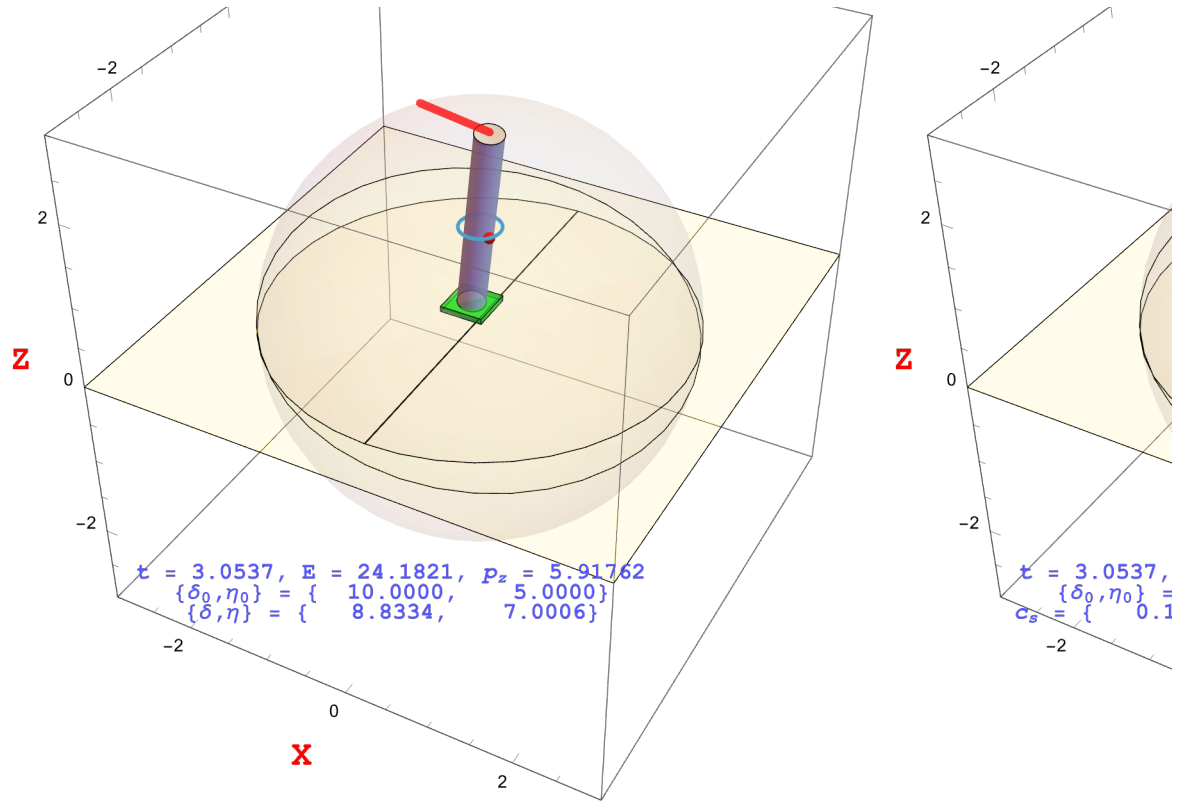
Side-by-Side Interactive Animation in $SO(3)$ and Euler-Lagrange in $SU(2)$

In[137]:=

`interactiveSideBySideAnimation[pureODEsLA]`

Out[137]=





Conclusion

In this notebook, we have extended the geometric derivation of the classical dynamics of the inverted spherical pendulum and the symmetrical top from $SO(3)$ gauge to $SU(2)$. By treating the orientation of a rigid body simply as a geodesic path on the 3-sphere manifold of $SU(2)$, we have systematically eliminated the coordinate singularities (gimbal lock) unavoidable in any Euler-angle representation. The side-by-side animations furnish numerical evidence of the superiority of the $SU(2)$ gauge choice, maintaining physical fidelity even at the poles where naive $SO(3)$ solvers fail. Furthermore, the numerical agreement between our “pure” geometric approach (solving for geodesics) and the traditional Lagrangian approach (Principle of Least Action) reinforces the mathematical unity of these two paths.

Future Work and Analytical Validation

While the numerical results are compelling, several areas remain for formal analytical verification and validation to complete the theory. These are marked throughout the text and summarized here as a roadmap for future development:

1. **Analytical Proof of Equivalence:** A formal derivation is needed to show that the geometrical equations of motion (geodesics) and the Euler-Lagrange equations (by Least Action) in $SU(2)$ are equivalent. Currently, we accept numerical evidence.
2. **Metric Stabilizer Derivation:** The “radial stabilizer” in the $SU(2)$ metric was determined by trial-and-error to ensure a non-singular, covariant inverse metric. A formal derivation of the optimal stabilizer term from first principles is required.
3. **The contravariant form of the Lagrange constraint force vector (cn4)** requires a more rigorous derivation to ensure it generalizes to arbitrary manifolds beyond S^3 .
4. **Algebraic Elaboration:** Further elaboration on the Lie Bracket $[A, B]$ and the specific conditions under which $\mathfrak{su}(2)$ fails to be a group under the bracket operation will be beneficial.
5. **Clifford Connections:** Mapping the 2×2 complex representations to spinors via Clifford algebra to fully bridge the gap between this classical treatment and upcoming quantum physics will be beneficial.

References

1. Beckman, Brian, *[SO\(3\) Gauge Theory for Classical Mechanics in UD](#)*. Wolfram Community post.
2. Beckman, Brian, *[From the steam age to quantum fields: a unified approach via UD tensorial calculus](#)*. Wolfram Community post.

3. Beckman, Brian. *General relativity in the UD calculus*. Wolfram Community post.
4. Beckman, Brian. *Formal Differential Geometry in the UD Calculus: Part 1*. Wolfram Community post.
5. Beckman, Brian. *Covariant Derivatives and Connections in the UD Calculus*. Wolfram Community post.
6. Beckman, Brian. *Riemann Curvature in UD + Compiler POC*. Wolfram Community post.
7. Beckman, Brian. *Schwarzschild Orbits in the UD Calculus*. Wolfram Community post.
8. Beckman, Brian. *Schwarzschild Metric from Scratch via UD Compilation*. Wolfram Community post.
9. Beckman, Brian. *Gauge Theory in UD*. Wolfram Community post.
10. Manfredo P. do Carmo, *Differential Geometry of Curves and Surfaces, Revised and Updated Second Edition*,
 Page 286, Poincaré's Theorem: *the sum of the indices of all singularities of a differentiable vector field with isolated singular points on a compact surface equals the Euler-Poincaré characteristic of the surface*. Because the Euler characteristic of a 2D sphere is $\chi(S^2) = 2$, which is not zero, it is impossible to draw a continuous tangent vector field without at least one singularity.
11. John Baez, Javier P. Muniain, *Gauge Fields, Knots and Gravity*, Part II, "Gauge Fields" and the sections on Fiber Bundles.
12. Jerrold E. Marsden, *Tudor Ratiu, with Ralph Abraham. Manifolds, Tensor Analysis, and Applications*, Third Edition (link unavailable).
 Theorem 8.5.12 (Hairy Ball Theorem). *Every vector field on an even-dimensional sphere has a critical point.*
13. William L. Burke, *Applied Differential Geometry*.